RĪGAS TEHNISKĀ UNIVERSITĀTE

Datorzinātnes, informācijas tehnoloģijas un enerģētikas fakultāte

Informācijas tehnoloģijas institūts

Egija Kokoreviča, Miks Lapsa

Akadēmisko bakalaura (pirmā cikla) studiju programmas

Informācijas tehnoloģija

studenti, stud. apl. nr. 221RDB288, 221RDB247

Apmācības metožu salīdzinājums šaha spēles aģenta realizācijai

BAKALAURA DARBS

Zinātniskais vadītājs *Dr. sc. ing.*, RTU profesors

Egons Lavendelis

RĪGA 2025

RĪGAS TEHNISKĀ UNIVERSITĀTE

DATORZINĀTNES, INFORMĀCIJAS TEHNOLOĢIJAS UN ENERĢĒTIKAS FAKULTĀTE

Informācijas tehnoloģijas institūts

Bakalaura darba izpildes lapa

Noslēguma darba autori:

studente Egija Kokoreviča _____

(paraksts, datums)

students Miks Lapsa _____

(paraksts, datums)

Noslēguma darbs ieteikts aizstāvēšanai:

Zinātniskais vadītājs:

Dr. sc. ing., RTU Profesors Egons Lavendelis _____

(paraksts, datums)

ANOTĀCIJA

Atslēgvārdi – MCTS, CNN, pašspēles, vēsturiskās spēlēs balstīta apmācība, šaha dzinis.

Bakalaura darbā tiek salīdzinātas divas apmācības metodes šaha spēles aģenta realizācijai – pašspēle un vēsturisko spēļu analīze. Darbā ir aplūkoti algoritmi, kas pielietoti šaha aģenta realizācijai un apskatīti šobrīd spēcīgākie šaha dziņi – Stockfish un Lc0. Papildus tam, tiek apskatīti arī dažādi neironu tīklu veidi un to pielietojums šaha spēles kontekstā. Darbojoties uz ierobežotiem skaitļošanas resursiem un izmantojot CNN neironu tīklu un MCTS pārmeklēšanas algoritmu, tiek izveidots pamats divu aģentu realizācijai. Apmācīto aģentu labākās versijas tiek salīdzinātas savā sarpā un noskaidrots, ka aģents, kurš apmācīts, izmantojot vēsturisko spēļu analīzi ir sasniedzis augstu meistarības līmeni, turpretī pašspēles aģents krietni atpaliek un nav tik konkurētspējīgs. Iemesls šai atšķirībai ir skaidrojams ar to, cik resursietilpīgi ir šādu aģentu izveide, kur, jo īpaši, pašspēles aģents aizņem daudz vairāk laika spēļu ģenerēšanas un apmācības procesā. Darba noslēgumā ir izstrādātas rekomendācijas apmācības metodes izvēlei, balstoties uz darbā veiktajiem eksperimentiem un rezultātiem.

Darba kopējais apjoms sastāv no 69 lappusēm, 14 attēliem, 4 tabulām, 1 pielikumu, un 50 izmantotajiem avotiem.

ABSTRACT

Keywords – MCTS, CNN, self-play, historical game analysis training, chess engine.

This Bachelor's thesis compares two training methods for the implementation of a chess-playing agent – self-play and the analysis of historical games. The thesis discusses algorithms used for chess agent implementation and examines the currently strongest chess engines – Stockfish and Lc0. Additionally, various types of neural networks and their application in the context of chess are also examined. Operating with limited computational resources and using a CNN neural network and the MCTS search algorithm, a foundation is established for the implementation of two agents. The best versions of the trained agents are compared against each other, and it is determined that the agent trained using historical game analysis has achieved a high level of mastery, whereas the self-play agent lags significantly behind and is not as competitive. The reason for this difference is attributed to how resource-intensive the creation of such agents is, where, in particular, the self-play agent consumes significantly more time in the game generation and training process. In conclusion, recommendations are developed for the choice of training method, based on the experiments and results conducted in the thesis.

The bachelor’s thesis contains 69 pages, 14 images, 4 tables, 1 appendix, and 50 references.

SATURA RĀDĪTĀJS

IEVADS 7
1. ŠAHA TEORĒTISKIE PAMATI UN ALGORITMISKIE RISINĀJUMI 9
1.1. Divpersonu spēles ar pilnu informāciju jēdziens 9
1.2. Šaha spēles koks un stāvokļu telpa 10
1.2. Minimax pārmeklēšanas algoritms 11
1.3. Heiristiskie novērtējumi 13
1.4. Alfa-Beta nogriešanas algoritms 14
1.5. Montekarlo pārmeklēšanas algoritms 16
1.6. Elo reitings 18
1.7. Stockfish 20
1.8. Leela Chess Zero 22
1.9. Algoritmu un metožu izvēle dziņa izstrādei uz ierobežotiem resursiem 23
2. APMĀCĪBAS METOŽU APSKATS 25
2.1. Mašīnmācīšanās jēdziens 25
2.2. Apmācības metodes divpersonu spēlēm ar pilnu informāciju 26
2.2.1. Uzraudzīta apmācība 26
2.2.2. Neuzraudzīta apmācība 27
2.2.3. Stimulēta apmācība 28
2.3. Metožu pielietojums šaha spēles kontekstā 29
2.4. Mākslīgo neironu tīklu definīcija 30
2.5. Neironu tīklu veidi 30
2.5.1. Konvolucionālais neironu tīkls 30
2.5.2. Transformatoru modeļi 32
2.5.3. Efektīvi atjaunināmais neironu tīkls 34
2.6. Neironu tīkli šaha spēles kontekstā 34
3. AĢENTA REALIZĀCIJA 36
3.1. Gājienu un spēles stāvokļu kodēšana neironu tīklam 36
3.2. Neironu tīkls 38
3.3. MCTS pārmeklēšanas algoritms ar PUCT 41
3.3.1. Aģentu veiktspējas testēšana ar MCTS 42
3.4. Eksistējošo spēļu aģents 45
3.4.1. Spēļu lasīšana 46

3.4.2. Aģenta apmācība 48
3.5. Pašspēles aģents 49
3.5.1. Spēļu ģenerēšana 49
3.5.2. Aģenta apmācība 49
3.6. Salīdzināšanas metodoloģija 50
3.7. Šaha aģentu salīdzinājuma rezultāti 52
3.8. Apmācības metožu salīdzinājums 55
3.9. Rekomendācijas apmācības metodes izvēlei 58
REZULTĀTI UN SECINĀJUMI 60
IZMANTOTIE INFORMĀCIJAS AVOTI 63
PIELIKUMI 68

IEVADS

Mūsdienās populārākie šaha spēles aģenti *Stockfish* un *Leela Chess Zero* ir sasnieguši tik augstu veiktspēju, ka cilvēki vairs nespēj ar tiem konkurēt (Adrien, 2024). Ņemot vērā šaha spēles sarežģītību, šo aģentu efektivitāte un apmācība ir balstīta uz algoritmiem, kas pieprasa milzīgu daudzumu skaitļošanas resursu izmantošanu (stockfishchess, 2025). Neskatoties uz to augsto līmeni, tos turpina attīstīt un uzlabot, tādā veidā pārbaudot mākslīgā intelekta spējas, testējot algoritmu darbību un kopumā sniedzot pienesumu datorzinātnēs, mākslīgā intelekta un spēļu izstrādes sfērās. Tomēr, šādi resursietilpīgi risinājumi nav pieejami visiem izstrādātājiem, kas noved pie situācijas, kad ar šāda līmeņa aģentiem, izstrādes un testēšanas nolūkos var strādāt tikai salīdzinoši maza kopiena. Turklāt, ņemot vērā faktu, ka populārākie spēles aģentu apmācības veidi ir pašspēle un vēsturisko spēļu analīze, nav veikta šo pieeju dziļāka izpēte un analīze salīdzināšanas nolūkos. Īpaši trūkst izpētes par to, kura pieeja būtu efektīvāka situācijās, kad pieejami ierobežoti skaitļošanas resursi. Šaha aģenta izstrāde, kas efektīvi darbotos uz pieejamiem ierobežotajiem skaitļošanas resursiem un pieejamā metodoloģija, kas palīdzētu izvēlēties piemērotāko apmācības metodi savam aģentam radītu iespēju šajā nozarē iesaistīties plašākam izstrādātāju klāstam.

Darba mērķis ir veikt uz pašspēlēm un uz vēsturiskām spēlēm balstītu mašīnmācīšanās pieeju salīdzinājumu šaha spēles aģenta izstrādei.

Darba uzdevumi:

1. Veikt divpersonu spēļu ar pilnu informāciju realizācijai izmantoto pieeju analīzi šaha spēles kontekstā.
2. Veikt apmācības metožu apskatu divpersonu spēļu ar pilnu informāciju spēlēšanas aģentu apmācības kontekstā.
3. Izstrādāt apmācības metožu salīdzināšanas metodoloģiju šaha spēles kontekstā.
4. Veikt eksperimentālu uz esošām spēlēm un uz spēlēšanu pašam ar sevi balstīto apmācības metožu salīdzinājumu šaha spēles dziņa izstrādei.
5. Izstrādāt rekomendācijas apmācības metodes izvēlei.

Pirmajā darba daļā tiek vispārīgi aprakstīta šaha spēles būtība. Tiek dots detalizēts ieskats šaha spēles koka sarežģītībā un stāvokļu telpas analīzē. Tālāk ir veikta dziļāka izpēte par tādiem algoritmiem, kā Minimax, Alfa-Beta un Montekarlo, kā arī dots ieskats par to, kā tiek veidota spēles gājienu heiristiskā novērtēšana. Nodaļas noslēgumā tiek aplūkoti divi populārākie un šobrīd progresīvākie šaha dziņi – Stockfish un Leela chess zero, sniedzot ieskatu to arhitektūrā un kā to sniegums ir progresējis laika gaitā.

Otrajā darba daļā tiek likts fokuss uz mašīnmācīšanās metodēm – apskatīti apmācības metožu veidi un to pielietojums šaha spēles kontekstā. Būtisks uzsvars likts uz mākslīgo neironu tīklu izpēti, to veidiem un pielietojumu.

Trešajā darba daļā tiek detalizēti aprakstīts šaha aģentu izstrādes process, sākot no vides uzstādīšanas, aģentu realizācijas un apmācības procesiem, līdz aģentu salīdzināšanas un eksperimentēšanas posmam. Gala reazultātā iegūti nepieciešamie dati, lai veiktu objektīvu salīdzinājumu starp apmācības metodēm un izstrādātas rekomendācijas apmācības metožu izvēlei.

1. ŠAHA TEORĒTISKIE PAMATI UN ALGORITMISKIE RISINĀJUMI

Šaha spēle jau gadsimtiem ilgi ir aizrāvusi cilvēkus ar tās spēju padarīt spēli ne tikai izklaidējošu, bet arī intelektuāli sarežģītu. Tā prasa īpašu koncentrēšanās spēju un iedziļināšanos stratēģiju veidošanā, spēju paredzēt pretinieka gājienus un analizēt dažādas pozīcijas. Šo iemeslu dēļ, šahs ir kļuvis populārs kā nozīmīgs izpētes objekts ne tikai šaha entuziastu pusē, bet arī datorzinātnē, kas to izmanto dažādu algoritmu izstrādē.

Šajā nodaļā ir apskatīti šaha spēles pamatprincipi, sākot ar nelielu ieskatu spēles teorijā. Tālāk turpinot iedziļināties spēles būtībā ar matemātisku pieeju, kas rezultātā noveda pie mūsdienās populārāko algoritmu apskata, sākot no klasiskākajiem piemēriem, kas ir Minimax un Alfa-Beta algoritms un turpinot ar jaunākām pieejām kā Montekarlo pārmeklēšanas algoritms un neironu tīkli. Tiks aplūkoti arī šobrīd spēcīgākie šaha spēles dziņi, kas būtiski atšķiras ar savu arhitektūru un darbības principiem – Stockfish un Leela Chess Zero.

Šīs nodaļas mērķis ir rast teorētisko ieskatu, kas novedīs pie secinājumiem un algoritmu izvēles, kas tiks izmantoti un veidos pamatu prakstiskajai daļai, kurā tiks izstrādāti šaha dziņi un salīdzināts to apmācības veids.

1.1. Divpersonu spēles ar pilnu informāciju jēdziens

Spēļu teorijā divpersonu spēle ar pilnu informācija ir spēle, kurā abiem spēlētājiem katrā gājienā ir pieejams pilns pašreizējais spēles stāvoklis, ir zināmi visi veiktie gājieni, kā arī gājieni ir secīgi (Shor, 2025). Šaha spēle atbilst šiem kritērijiem, jo ir zināmi visi abu spēlētāju veiktie gājieni, turnīra formātā tos pierakstot ar īpašu notāciju, tiem tiekot secīgi veiktiem, padarot to par secīgu divpersonu spēli ar pilnu informāciju. Papildus tam ir zināmi visi iespējamie sekojošie gājieni, kādus pretinieks varētu veikt, vadoties pēc šaha spēles noteikumiem, kas ierobežo iespējamo gājienu skaitu, atkarībā no pašreizējā spēles stāvokļa – ir iespējams izveidot ļoti lielu spēles koku ar visiem iespējamiem gājieniem pat pirms pirmā veiktā gājiena.

Viens no nepilnas informācijas spēles piemēriem ir Uno, kur katram spēlētājam ir redzamas tikai un vienīgi savas kārtīs – nav pieejams pilns pašreizējais spēles stāvoklis, jo nav zināms, kādus gājienus būs iespējams veikt citiem spēlētājiem.

Pilnas informācijas spēles ir interesantas mašīnmācīšanas un dažādu apmācības metožu pielietošanas ziņā, jo ir iespējams precīzi novērtēt pašreizējo spēles stāvokli un veikt tālāku gājienu analīzi, palīdzot novest pie vislabākā rezultāta. Šaha gadījumā šo novērtējumu sarežģī iespējamais kombināciju skaits, it īpaši spēles sākumā – Šanona skaitlis (Shannon, 1950).

1.2. Šaha spēles koks un stāvokļu telpa

Šahs ir divpersonu spēle, kuras pamatā ir mērķis uzvarēt pretinieku, kas notiek gadījumā, kad pretinieka spēles figūra "Karalis" ir nostādīts pozīcijā, kur tas vairs nespēj pārvietoties uz drošu lauciņu un ir pakļauts uzbrukumam. Spēles laukuma izmērs ir 8x8 lauciņi un katra spēlētāja rīcībā ir savas 16 figūras. Šaha spēli sarežģītu padara figūru dažādie pārvietošanās veidi pa spēles laukumu. Pašā spēles sākumā, katram spēlētājam ir 20 dažādi gājieni, kā uzsākt spēli, jeb pārvietot kādu no savām figūrām spēles laukumā. Atkarībā no tā, kādā spēles pozīcijā spēlētājs atrodas, vidēji ir iespējams izdarīt 30 dažādus gājienus (Shannon, 1950).

Par spēles stāvokli sauc konkrēto pozīciju vai situāciju, kurā spēle šobrīd atrodas un to raksturo visa nepieciešamā informācija, lai pilnībā aprakstītu spēli (Filimowicz, 2023). Šaha spēles kontekstā šī raksturojošā informācija sevī iekļauj, piemēram, figūru izkārtojumu uz spēles laukuma, spēles laiku, gājienu vēsturi, kuram spēlētājam ir nākamais gājiens un spēles beigu rezultātu.

Stāvokļa telpas sarežģītība ir definēta kā likumīgo spēles pozīciju skaits, ko var iegūt no sākotnējās pozīcijas spēlē (Heule & Rothkrantz, 2007). Sakarā ar to, cik daudz dažādi spēles varianti ir iespējami un šaha noteikumu sarežģītības dēļ, piemēram, bandinieka pārvēršanās par citu figūru gadījumā, kad paveras jaunas iespējas figūras pārvietošanai pa galdiņu, nav iespējams precīzi aprēķināt skaitli, kas noteiktu visu legālo pozīciju skaitu spēles laikā. Aptuvenais skaitlis šaha spēles legālajām pozīcijām, kas ir iegūts ir 10^{40} (Shannon, 1950).

Spēles koks, kas atšķirībā no stāvokļu telpas, ir visu iespējamo gājienu attēlojums no spēles sākuma līdz spēles beigām, kas sevī ietver arī identiskas spēles pozīcijas, kas

tika iegūtas caur dažādiem gājieniem. Tāpat kā ar stāvokļu telpu, arī spēles koka lielumam nav iespējams iegūt precīzu skaitli, taču ir iegūts aptuvens skaitlis 10^{120}, ko mūsdienās dēvē arī par Šanona skaitli (*Shannon number)* (Shannon, 1950). Iepriekšminētajā avotā tiek noskaidrots, ka šis skaitlis tika aprēķināts, pieņemot, ka vidēji jebkādā spēles pozīcijā ir iespējams izdarīt izvēli starp 30 dažādiem gājieniem un katrs spēlētājs visas spēles laikā veic vidēji 40 gājienus.

Ņemot vērā to, cik plaša ir gājienu izvēle, rodas nepieciešamība pēc algoritma, kas spēs atrast optimālus ceļus un gājienus, neskatoties uz to, ka spēles koka pārmeklēšana ir ierobežota. Kā viens no šādiem algoritmiem ir Minimax pārmeklēšanas algoritms, kas darbojās kā pamats daudzām klasiskām šaha programmām, kā arī uz tā pamata tika izstrādātas optimizētas versijas, no kurām ievērojamākā ir Alfa-Beta algoritms.

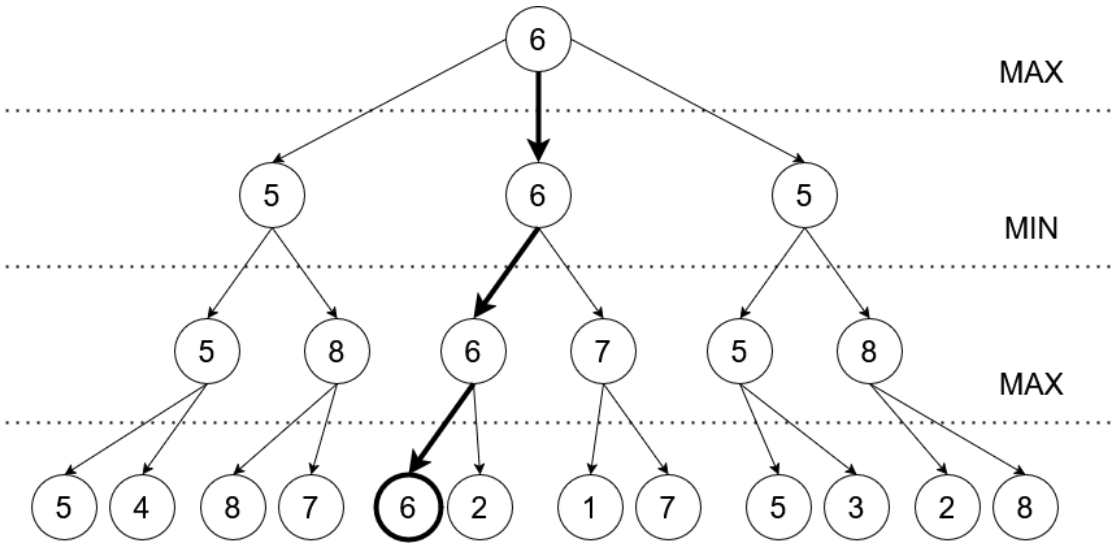
1.2. Minimax pārmeklēšanas algoritms

Minimax pārmeklēšanas algoritma darbības pamatā ir mērķis aplūkot visus iespējamos gājienus spēles kokā līdz noteiktam koka dziļumam (Wilkens & Khoury. 2024). Atsaucoties uz 1.1 nodaļu, kurā tika noskaidrots, ka šaha spēles koka lielums ir apmēram 10^{120}, un zarošanās koeficients, jeb iespējamo gājienu skaits no katras pozīcijas ir vidēji 30, tad varam secināt, ka pārmeklējot koku pat nelielā dziļumā, tiks patērēts milzīgs daudzums skaitļošanas resursu. Šī iemesla dēļ, Minimax algoritms pieder pie tā sauktajiem pilnas pārmeklēšanas algoritmiem, kas neatkarīgi no šķietami neveiksmīgiem un neefektīviem gājieniem, aplūkos visus iespējamos variantus, lai atrastu labāko ceļu starp tiem, kas potenciāli varētu novest pie uzvaras, vai sliktākajā gadījumā, kad uzvara nav iesējama, dosies uz neizšķirtu rezultātu (Wilkens & Khoury, 2024).

Lai Minimax algoritms saprastu, kuru ceļu izvēlēties kā labāko, ir nepieciešams saprast spēles noteikumus un nosacījumus, kas stājas spēkā uzvaras, zaudējuma, vai neizšķirta rezultāta gadījumos. Katram spēles stāvoklim, kuru algoritms spēles kokā apskata, tiek piešķirts skaitlisks vērtējums, kas nosaka spēles iespējamo iznākumu. Algoritms spēlētājiem piešķir nosaukumus “maksimizētājs” un “minimizētājs”, no kuriem maksimizētājs vienmēr tieksies doties pa ceļu, kas ir novērtēts ar augstāko iespējamo vērtību, savukārt minimizētājs meklēs virsotnes ar zemāko iespējamo

vērtējumu (Shevtekar, Malpe, Bhaila 2022). Piemēram, ja spēles stāvoklis tiek novērtēts ar "1", tad tas nozīmēs, ka maksimizētājs ir vadībā un tiecas uz uzvaru, turpretī spēles stāvoklis ar vērtējumu "-1" liecinās par minimizētāja uzvaru, bet vērtējums "0" nozīmētu neizšķirtu spēles rezultātu. Algoritms ir rekursīvs, kas nozīme, ka tas spēles stāvokļu novērtēšanu sāk no spēles koka beigām un pieņem, ka abi spēlētāji veiks savu gājienu izvēli perfekti (Calderan, 2020). Situācijā, kad spēle ir vienkārša un spēles koks ir pavisam neliels, algoritms ir spējīgs ātri aplūkot visu spēles koku no sākuma līdz spēles beigām. Šādā gadījumā, tas sāks savu novērtēšanu no koka beigām, kur jau ir zināmi visi spēles rezultāti, un atkāpsies novērtējot katru gājienu, kas veda pie šiem rezultātiem.

Uzskatāmi Minimax algoritma darbības piemēru ilustrē 1.1. attēls, kur ir attēlots kādas spēles koks ar 4 līmeņiem. Attēla labajā pusē norādīts, kurš no spēlētājiem maksimizētājs (max), vai minimizētājs (min) veiks gājienu. Algoritms darbību sāks no koka beigām, kur maksimizētājam ir gājiens, un tas apskatot visus pēctečus izvēlēsies lielāko vērtību starp tiem, savukārt minimizētājs sava gājiena laikā izvēlēsies mazāko vērtību no dotajām. Turpinot atpakaļizplatīšanas procesu tiek novērtētas visas koka virsotnes. Gala rezultātā iespējams noskaidot, kādu ceļu ies maksimizētājs un pie kādas vērtības, jeb rezultāta tas nonāks, pieņemtot, ka pretinieks spēlēs optimāli. Uzvaru nesošais ceļš attēlā ir iezīmēts ar izceltām, tumšām līnijām.



1.1. Vienkāršs Minimax algoritma darbības piemērs

Šaha spēles kontekstā šis nav tik vienkārši – ņemot vērā faktu, ka vienlaicīgi aplūkot visu spēles koku nav iespējams, nākas noteikt konkrētu dziļumu, kādu tas novērtēs. Pieņemsim, ka tiek izvēlēts apskatīt spēles koku 3 gājienus uz priekšu - rodas situācija, kurā spēles rezultāts vēl nemaz nav zināms, kas noved pie nepieciešamības izplānot, kā spēles stāvokļus novērtēt spēles vidū.

1.3. Heiristiskie novērtējumi

Lai algoritms saprastu, kādus gājienus izvēlēties, ir nepieciešams katram no tiem piešķirt vērtējumu, kas liecinās par to efektivitāti. Kā viens no vienkāršākajiem veidiem ir skatīties uz figūru vērtībām, kas ir jau standartizētas un liecina par to spēku, jeb nozīmīgumu, kur bandiniekam vērtība ir 1 punkts, zirgam un laidnim 3, tornim 5, karalienei 9 un karalim vērtība ir bezgalīga, vai netiek piešķirta, jo tā zaudēšana automātiski liecina par spēles beigām. Izmantojot šo novērtēšanas funkciju, ir iespējams izpildīt labus figūru apmaiņas gājienus, kurā pretinieks zaudē figūru ar lielāku vērtību, vai tieši pretēji, spēt izvairīties no neizdevīgiem figūru apmaiņas gadījumiem, vai vienkārši figūras zaudēšanu gadījumā, ja turpmākie gājieni nedos pārspēku (Calderan, 2020).

Kā nākamo heiristiskās novērtēšanas veidu varētu pieminēt figūru-lauciņu tabulu (*piece-square table)*, kurā katrai unikālai figūrai uz spēles laukuma kvadrātiem tiek piešķirts skaitlisks novērtējums, kurš nosaka, cik izdevīga pozīcija tā ir šai figūrai (Calderan, 2020). Uzskatāmi ilustrē 1.2. attēlā dotais heiristiskais novērtējums spēles lauciņiem zirga figūrai, kurā ar augstāko pozitīvu skaitli tiek apzīmēta visizdevīgākā pozīcija, savukārt ar zemāko negatīvo skaitli attēlota neizdevīgākā pozīcija. Ņemot vērā zirga figūras unikālo pārvietošanās veidu pa spēles laukumu, tam attiecīgi ir izdevīgāk atrasties spēles laukuma vidū, kurā tam paveras plašākas pārvietošanās iespējas, nevis būt nostādītam laukuma stūrī.



```
[-5.0, -4.0, -3.0, -3.0, -3.0, -3.0, -4.0, -5.0]
[-4.0, -2.0,  0.0,  0.0,  0.0,  0.0, -2.0, -4.0]
[-3.0,  0.0,  1.0,  1.5,  1.5,  1.0,  0.0, -3.0]
[-3.0,  0.5,  1.5,  2.0,  2.0,  1.5,  0.5, -3.0]
[-3.0,  0.0,  1.5,  2.0,  2.0,  1.5,  0.0, -3.0]
[-3.0,  0.5,  1.0,  1.5,  1.5,  1.0,  0.5, -3.0]
[-4.0, -2.0,  0.0,  0.5,  0.5,  0.0, -2.0, -4.0]
[-5.0, -4.0, -3.0, -3.0, -3.0, -3.0, -4.0, -5.0]
```

1.2. att. Zirga figūras heiristiskais pozīcijas novērtējums uz spēles galda lauciņiem (aizgūts no (Hartikka, 2017))

Šī novērtējuma funkcija var kļūt vēl efektīvāka, ja to pielāgo atkarībā no esošā spēles stāvokļa. Piemēram, ja pašā spēles sākumā karaļa figūru ir izdevīgi turēt pasargātu savos spēles galda puses stūros, tad spēles beigu daļā, kad karali vairs neapdraud tik daudzas figūras, tad kļūst izdevīgi to lietot kā aktīvu figūru un tieši pretēji doties uz spēles galda centru (Calderan, 2020).

Vēl viens novērtēšanas veids ir saistīts ar to, cik ātri ir iespējams panākt matu, jeb uzvaru spēlē. Gadījumā, kad algoritms redz ceļus, kas noved pie uzvaras, tad, jo ātrāk šī uzvara tiek panākta, jo vērtīgāks būs šis ceļš (Calderan, 2020). Tomēr, ar šo pieeju vajadzētu būt uzmanīgiem un nevajadzētu jaukt šaha pieteikšanu pretinieka karalim ar garantētu potenciālu matu. Jo īpaši spēles beigās, kad uz laukuma ir palicis neliels skaits figūru, šaha dzinis, kuram ir iemācīts, ka šaha pieteikšana pretiniekam ir lielisks gājiens, tad tas var novest pie situācijas, kurā nepārdomāti tiek likts fokuss uz tikai uz to. Rezulātā, novārtā paliek atstātās pārējās pieejamās figūras un veikti bezjēdzīgi gājieni ar vienu figūru sekojot un mēģinot pieteikt šahu pretinieka karalim. Līdz ar to nozīmīgs uzsvars būtu jāliek arī uz pozīcijas stratēģisko kvalitāti ilgtermiņā, nevis tiekties uz īslaicīgu priekšrocību.

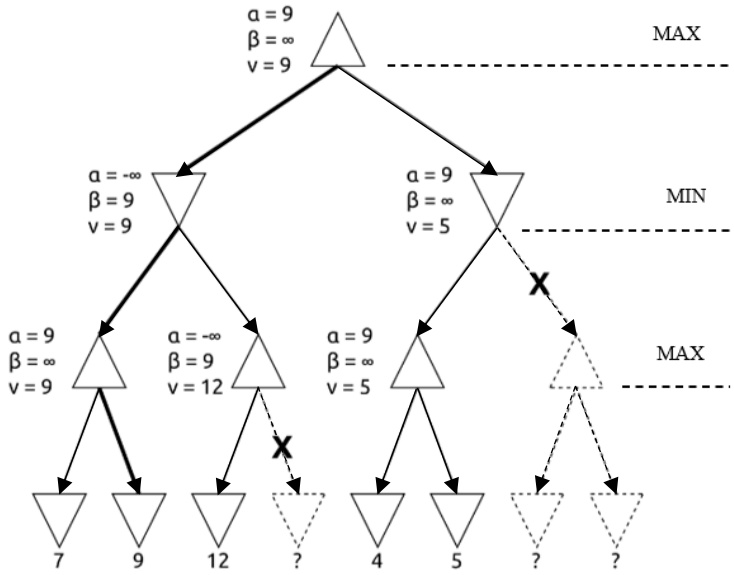
1.4. Alfa-Beta nogriešanas algoritms

Kā papildinājums un efektivitātes uzlabojums jau iepriekšminētajam Minimax algoritmam ir alfa-beta nogriešanas algoritms. Tā darbības pamatā ir mērķis nogriezt

jeb izslēgt no turpmākas apskatīšanas koka zarus, virsotnes, kas vairs neved pie labāka rezultāta kā tas, kurš jau ir izskatīts. Algoritms būtiski samazina izskatīto spēles koka virsotņu skaitu, turklāt atgriež identisku rezultātu kādu Minimax algoritms būtu devis (Calderan, 2020). Šīs darbības rezultātā tiek ievērojami samazināts laiks, kas nepieciešams spēles koka izskatīšanai un tā iemesla dēļ, ir iespējams pārmeklēt koku dziļākos līmeņos.

Algoritms ievieš divus jaunus parametrus, kas ir attēloti jau tā nosaukumā – alfa un beta. Alfa parametrs attēlo labāko, jeb augstāko vērtību, ko maksimizētājs var iegūt, turpretī beta parametrs attēlo zemāko vērtību, ko minimizētājs var gūt (Marckel, 2017). Tāpat, kā Minimax, arī alfa-beta algoritms ir rekursīvs un apskata koka virsotnes sākot no beigām un kreisās puses, pārnesot heiristiskos vērtējumus atpakaļ līdz saknes virsotnei. Spēles koka zaru, virsotņu nogriešana notiek gadījumā, kad stājas spēkā nosacījums, kur alfa vērtība ir vienāda, vai lielāka par beta vērtību (Marckel, 2017).

Algoritma darbības piemērs ir vizualizēts 1.3. attēlā. Alfa un Beta parametri sākotnēji tiek inicializēti ar vērtībām ∞ un -∞, kas nodrošina to, ka šīs vērtības garantēti tiks aizvietotas ar labākiem vērtējumiem, kas tiks atrasti pārmeklējot spēles koku (Marckel, 2017). Attēlā redzamā v vērtība ir tā, ko sava gājiena laikā maksimizētājs, vai minimizētājs ir izvēlējies kā labāko heiristisko vērtību, apskatot esošās virsotnes pēctečus.



1.3. Alfa-Beta nogriešanas algoritma piemērs (modificēts no Marckel, 2017))

1.5. Montekarlo pārmeklēšanas algoritms

Kā vēl viens pārmeklēšanas algoritma piemērs, kurš būtiski atšķiras ar savu darbības principu, salīdzinot ar iepriekš apskatītajiem algoritmiem, ir Montekarlo, kura pamatā ir gājienu heiristiskā vērtējuma noteikšana pēc nejaušas izvēles gājienu simulācijas (Browne, Powley, et al., 2012). Algoritma darbību var iedalīt četros galvenajos soļos (Wilkens & Khoury, 2024) un tie ir šādi:

1) Selekcija: Process sākas no saknes virsotnes, kas ataino esošo spēles stāvokli. Algoritms sāk izskatīt virsotnes pēctečus, jeb nākamos iespējamos gājienus un izvēlas vienu no tiem, kuru izvērst plašāk, izmantojot Augšējās ticamības robežu, kas pielietota spēles kokiem, jeb saīsināti no angļu valodas UCT - *Upper Confidence Bound Applied to Trees* (turpmāk UCT).
2) Paplašināšanās: No iepriekšējā posmā izvēlētās virsotnes, tiek nejaušā kārtā izvēlēts nākamais iespējamais gājiens, jeb viens no šīs virsotnes pēctečiem. Šī jaunā virsotne tiek saglabāta spēles kokā un turpmāk tiks izmantota atkārtotiem selekcijas procesiem.
3) Simulācija: Algoritms sāk simulēt turpmāko spēles gaitu, izvēloties nākamos iespējamos gājienus pēc nejaušas izvēles principa, gan sev, gan pretinieka vietā līdz brīdim, kad ir sasniegts noteikts spēles stāvoklis.
4) Atpakaļizplatīšana: Spēles stāvoklis, kas tika sasniegts simulācijas procesā tiek novērtēts ar skaitli, kas nosaka tā iespēju tiekties uz uzvaru, jeb būt pārsvarā pār pretinieku. Pēc tam šis vērtējums tiek nests atpakaļ uz spēles koka augšu, un vērtējums tiek piešķirts tikai tai virsotnei, no kuras tika sākta simulācija.

Montekarlo algoritms selekcijas posmā rada dilemmu, jo ir nepieciešams veikt izvēli starp virsotnēm, kas jau no iepriekšējām iterācijām un simulācijas procesiem ir parādījušas daudzsološus rezultātus un starp tām, kas vēl nav iepriekš izpētītas, taču potenciāli varētu dot vēl labāku rezultātu (Browne, Powley, et al., 2012). UCT izvēles princips tiek izmantots, lai ieviestu balansu starp šīm izvēlēm un šī formula (Wilkens & Khoury, 2024) izskatās šādi:

$$UCT = \bar{X_j} + C_p \sqrt{\frac{2\,ln\,N}{n_j}}, \qquad (1.1)$$

kur $\bar{X}_j$ – vidējā vērtība, kas ataino to, cik labs ir šis virsotnes pēcteča *j* gājiens;
C_p – parametrs, kas darbojas kā svars un nosaka to, cik ļoti algoritms tieksies uz jaunu virsotņu izpēti;
N – attēlo skaitu, cik bieži algoritms ir apmeklējis, jeb sācis simulāciju no šīs virsotnes;
n_j – attēlo skaitu, cik bieži algoritms ir apmeklējis, jeb sācis simulāciju no šīs virsotnes pēcteča.

Atgriežoties pie dilemmas situācijas, tad formulā (1.1) redzamā parametra $\bar{X}_j$ vērtība ir tā, kas liek algoritmam fokusēties uz veiksmīgu gājienu turpmāku izpēti, lai potenciāli vēl vairāk uzlabotu to rezultātu (Wilkens & Khoury, 2024). Attiecīgi – jo augstāka būs šī parametra vērtība, jo lielāka iespēja, ka algoritms šo virsotni liks kā prioritāti turpmāko simulāciju veikšanai.

Parametrs C_p ir tas, kurš noteiks, cik svarīgi algoritmam ir prioritizēt vēl neizpētītu gājienu simulāciju. Šo parametru lietotājs izvēlas un ieliek formulā kā konstanti pēc saviem ieskatiem.

Lai gan šis tradicionālais MCTS algoritms ir pierādījis savu spēju veiksmīgi risināt dažādus sarežģītas dabas uzdevumus, tomēr tā darbības princips, kura pamatā ir nejaušas izvēles gājienu simulācija, prasa daudz laiku un skaitļošanas resursus. Sarežģītākās spēlēs, piemēram, šahā, kur stāvokļu telpa ir milzīga, šāds risinājums varētu kļūt nepraktisks, ņemot vērā iespēju, ka nejauši simulēti gājieni ne vienmēr spēs attēlot optimālu spēli, kā rezultātā tas novedīs pie tā, ka algoritms patērē laiku izpētot neefektīvus ceļus un pārāk lēni nonāk pie optimāla risinājuma. Saskaršanās ar šāda tipa problēmām veicināja uzlabotu MCTS algoritma versijas izstrādi – Prognozētāja + Augšējās ticamības robežu, kas pielietota spēles kokiem, jeb saīsināti no angļu valodas PUCT - *Predictor + Upper Confidence Bounds for Trees* (turpmāk PUCT).

Atšķirībā no tradicionālā UCT izvēles principa, kas tika aprakstīts iepriekš, šajā gadījumā nāk klāt prognozētājs, jeb neironu tīkls, kas palīdz veikt nākamo gājienu izvēli un novērtēšanu un šī formula (Cazenave, 2024) izskatās šādi:

$$\text{PUCT} = Q(s,a) + c_p \cdot P(s,a) \cdot \frac{\sqrt{N(s)}}{1+N(s,a)}, \qquad (1.2)$$

kur Q(s,a) - attēlo skaitlisku vērtību, jeb to, cik labs ir gājiens *'a'*, kas tiek veikts no esošās pozīcijas *'s'*;

c_p – hiperparametrs, kas kontrolē to, cik ļoti algoritms tieksies uz jaunu, vēl neapskatītu virsotņu izpēti;

P(s,a) – skaitliska vērtība, kas attēlo cik labs ir gājiens *'a'*, kas tiek veikts no esošās pozīcijas *'s'*, ko ir noteicis neironu tīkls;

N(s) – attēlo skaitu, cik bieži algoritms ir apmeklējis, jeb sācis simulāciju no šīs virsotnes pēcteča;

N(s,a) - skaitlis, kas parāda cik reizes iepriekšējās simulācijās gājiens *'a'* ir veikts, jeb apskatīts no esošās pozīcijas *'s'*.

Neironu tīkla ieviešana šajā algoritmā nodrošina to, ka potenciāli veiksmīgi gājieni tiek jau izpētīti agri sākotnējās simulācijās un netiek patērēts laiks nejauši izvēlētu gājienu simulēšanai. Tas būtiski samazina veikto simulāciju skaitu un attiecīgi arī izmantotos skaitļošanas resursus. Tomēr, šajā gadījumā jāpaļaujas uz to, ka izveidotais neironu tīkls apmācības rezultātā spēj dot jēgpilnus un veiksmīgus gājienus, jo pretējā gadījumā, algoritms lieki tērēs laiku neveiksmīgu gājienu izpētē.

1.6. Elo reitings

Lai objektīvi novērtētu šaha spēlētāju prasmju līmeni, ungāru-amerikāņu fiziķis un šaha entuziasts Alprads Elo izstrādāja metodi, kas ar matemātisku pieeju ļauj aprēķināt spēlētāju reitingu, kas mūsdienās ir pazīstama kā Elo reitings (jayspeakman, n.d.). Kā apgalvo iepriekš minētā avota autors, pirms Elo reitinga sistēmas ieviešanas, spēlētāju reitingi tika likti patvaļīgi un nebija objektīvi, kas kļuva par iemeslu 1970. gadā Starptautiskajai šaha federācijai (FIDE - *Fédération Internationale des Échecs*) atzīt Elo reitingu par oficiālu vērtēšanas sistēmu. Šis reitings ir skaitlis, kas ataino spēlētāja prasmju līmeni, un atbilstoši šim skaitlim, spēlētāji tiek iedalīti kategorijās (1.1. tabula).

1.1. tabula

Spēlētāju iedalījums kategorijās (aizgūts no (Chandan, n.d.))

Reitinga diapazons	Kategorija
2700+	Augstākais Lielmeistaru līmenis, ieskaitot Pasaules līmeņa čempionus un Pasaules čempionāta kandidātus
2500–2700	Lielmeistari
2400-2500	Starptautiskie Meistari un Lielmeistari
2300-2400	FIDE Meistari un Starptautiskie Meistari
2200-2300	FIDE Meistara Kandidāti un Nacionālie Meistari
2000-2200	Meistara Kandidāti
1800-2000	A klase, 1. kategorija
1600-1800	B klase, 2. kategorija
1400-1600	C klase, 3. kategorija
1200-1400	D klase, 4. kategorija
Zem 1200	Iesācēji

Šobrīd pasaulē visaugstāk novērtētais šaha spēlētājs ar reitingu 2833 klasiskajā spēlē ir Magnuss Kārlsens (FIDE, 2025).

Elo reitings attēlo ne tikai spēlētāja prasmes, bet arī iespēju uzvarēt spēlē pret pretinieku un šīs iespējamības aprēķināšanai tiek izmantota formula (Glickman & Jones, 1999), kas izskatās šādi:

$$E = \frac{1}{1+10^{-(R_A - R_B)/400}}, \qquad (1.3)$$

kur R_A – pirmā spēlētāja (A) pašreizējais reitings;
R_B – otrā spēlētāja (B) pašreizējais reitings;
E – aprēķināmā iespējamība spēlētājam A uzvarēt šo spēli pret pretinieku B.
Pieņemsim, ka A spēlētājam pašreizējais reitings ir 900, savukārt B spēlētājam reitings ir 1150, tad pielietojot formulu (1.3) varam aprēķināt, ka A spēlētāja izredzes uzvarēt šo spēli, būs apmēram 0.19 jeb 19%.

Elo reitings abiem spēlētājiem tiek atjaunots balstoties uz iepriekšējās spēles rezultātiem ar formulu (Glickman & Jones, 1999):

$$r_{post} = r_{pre} + K(S - S_{exp}), \quad (1.4)$$

kur r_{pre} – spēlētāja esošais reitings;

K – koeficients k, jeb svars, kas mainās atkarībā no nosacījumiem, kur K=40, ja spēlētājs ir iesācējs un nav izspēlējis vairāk kā 30 spēles kā arī spēlētājiem, kas nav sasnieguši 18 gadu vecumu un to reitings ir zem 2300, K=20, ja spēlētāja reitings ir zem 2400 un K=10, ja spēlētājs ir sasniedzis reitingu 2400 un paliek šajā līmenī, pat, ja pēc tam nokrītas zem 2400 (FIDE, 2025);

S – spēlētāja rezultāts spēles beigās, kur S=1, ja spēlētājs uzvarēja, S=0.5, ja rezultāts bija neizšķirts un S=0, ja spēlētājs zaudēja spēli;

S_{exp} – iespējamība uzvarēt spēli pret pretinieku, kas ir aprēķināma ar iepriekšminēto (1.3) formulu;

r_{post} – spēlētāja jaunais reitings.

Aplūkojot (1.4) formulu iespējams nonākt pie secinājumiem, piemēram, gadījumā, kad nosacījums ($S - S_{exp}$) ir pozitīvs, tad tas norāda uz to, ka spēlētājs ir parādījis labāku rezultātu, nekā bija sagaidāms, kas attiecīgi palielinās viņa Elo reitingu, savukārt negatīva rezultāta gadījumā, spēlētājam reitings samazināsies (Glickman & Jones, 1999). To papildina k-faktora vērtība, jo tā ietekmēs to, cik strauji spēlētājam mainīsies Elo reitings – jo lielāka būs K vērtība, jo straujāk vērtējums var kristies, vai pieaugt.

Neskatoties uz to, ka šo vērtēšanas sistēmu joprojām izmanto un atzinusi par oficiālu FIDE, tā tomēr ir salīdzinoši veca un matemātiski vienkārša. Šobrīd dažādas šaha federācijas un šaha serveri izmanto dažādas vērtēšanas sistēmas, starp kurām populārākās, ieskaitot Elo, ir Glicko-1 un Glicko-2 (lichess, n/d). Balstoties uz iepriekšminēto avotu, šīs jaunākās sistēmas atšķiras no tradicionālā Elo ar to, ka tās ir matemātiski sarežģītākas un precīzākas, kā arī jaunajiem spēlētājiem uzreiz tiek piešķirts vērtējums 1500.

1.7. Stockfish

Stockfish ir atvērtā pirmkoda šaha dzinis, kas tika radīts 2008. gadā, un aktīvie uzlabojumi, kas joprojām tiek veikti rezultējas ar to, ka tas šobrīd ir viens no spēcīgākajiem šaha dziņiem pasaulē (Stockfishchess, n/d.). Iepriekš apskatītajā avotā noskaidrots, ka šobrīd jaunākā un aktuālā versija ir Stockfish 17, kas tika izlaista 2024-

09-06. Ņemot vērā, ka Stockfish tiek nepārtraukti attīstīts un pilnveidots jau 17 gadu garumā, tas padara to par vērtīgu izpētes objektu mākslīgā intelekta un mašīnmācīšanās jomās.

Līdz pat Stockfish 12 versijai, šis šaha dzinis nākamo gājienu ģenerēšanai izmantoja Alfa-Beta nogriešanas algoritmu un heiristisko novērtēšanas funkciju, kas tika izveidota ņemot vērā tādus faktorus, kā figūru vērtība un pozīcija uz laukuma, spēles stāvoklis u.c. (Sadmine, Husna & Muller, 2024). Tajā periodā Stockfish zināšanas balstījās tikai uz cilvēka doto sapratni par spēli, gājienu novērtēšanu un to efektivitāti.

Sākot no 12. versijas, Stockfish izmanto efektīvi atjauninātu neironu tīklu, jeb saīsināti no angļu valodas NNUE - *efficiently updatable neural network,* kas aizstāj manuāli veidotās heiristiskās novērtēšanas funkciju, kuras tika izmantota iepriekš (Sadmine, Husna & Muller, 2024). Neironu tīkla ieviešana samazina nepieciešamību pēc tik biežām manuālām korekcijām heiristisko funkciju vērtībās, tādā veidā atvieglojot izstrādātāju darbu. Stockfish sāka savu apmācības procesu balstoties uz eksistējošo spēļu lasīšanas un mācīšanās no tām (official-stockfish, n.d).

Paļaujoties uz neironu tīkla spēju mācīties no izspēlētām partijām un dažādu sarežģītu likumsakarību atrašanu, Stockfish ir iemācījies veikt daudz pārdomātākus un reizēm pat šķietami nestandarta gājienus, kas nes augstus rezultātus. Salīdzinot šo jauno versiju mačos pret iepriekšējo Stockfish 11 versiju, Stockfish 12 spēlēja ievērojami labāk un parasti uzvarēja vismaz 10 reizes vairāk spēļu pāru, nekā zaudēja, kur spēļu pāris atbilst izspēlētam vienam mačam gan ar melnajām, gan baltajām figūrām (stockfishchess, 2020). Tas pierāda to, ka balstoties tikai uz manuāli veidotām heiristikas funkcijām, ir sarežģītāk atrast stratēģiskus gājienus, jo cilvēku ieliktās zināšanas var izrādīties nepilnīgas un nepietiekamas. Turklāt, analizējot miljoniem partiju, neironu tīkls spēj novērtēt pozīcijas ar augstu precizitāti un atrast sīkas un nozīmīgas saistības kas cilvēkam varētu nebūt acīmredzamas.

Skatoties no skaitļošanas resursu puses, tad Stockfish galvenokārt balstās uz jaudīga procesora izmantošanu, jo NNUE uz tā darbojoties spēj ļoti efektīvi un ātri veikt pozīciju novērtēšanu spēles laikā (official-stockfish, n.d). Savukārt, kā minēts iepriekš apskatītajā avotā, videokarte tiek izmantota neironu tīkla apmācības procesā, pateicoties tā efektīvai spējai apstrādāt paralēlus procesus, kas ir īpaši noderīgi apstrādājot lielas datubāzes ar spēlēm.

1.8. Leela Chess Zero

Pēc tam kā uzņēmums DeepMind, kurš izstrādāja AlphaZero, šaha aģentu balstītu uz neironu tīklu, publicēja tā veiktspēju un mākslīgā intelekta efektivitāti sabiedrībai, tas radīja plašu interesi no datorzinātnieku, mākslīgā intelekta pētnieku un šaha entuziastu puses (Klein, 2022). 2017. gadā norisinājās un tika publicētas 100 spēles starp tā brīža vadošo šaha dzini Stockfish un AlphaZero, no kurām uz neironu tīkliem balstītais dzinis pārspēja Stockfish ar iespaidīgu rezultātu - 28 uzvaras un 72 neizšķirti no 100 spēlēm, neradot nevienu zaudējumu (Georgyan, 2023). Šis notikums demonstrēja mākslīgā intelekta spējas gūt pārākumu pār tradicionāliem pārmeklēšanas algoritmiem un heiristiskām novērtēšanas funkcijām, kā arī parādīja neironu tīku priekšrocību šāda tipa sarežģītu problēmu risināšanā. Tomēr jāņem vērā fakts, ka salīdzinājums starp Stockfish un AlphaZero nav pilnībā godīgs, zinot to, ka AlphaZero darbojās uz Goolge superdatora ar ievērojamiem skaitļošanas resursiem, savukārt Stockfish tolaik balstījās uz daudz ierobežotākiem resursiem (Schulz, 2017). AlfaZero izmantoja stimulētās apmācības metodi, kurā neironu tīkls izvērtēja stratēģiskas pozīcijas un veica lēmumus turpmākajiem gājieniem, balstoties uz iepriekšējo spēļu rezultātiem, kā arī PUCT algoritmu, kas būtiski samazināja pārmeklēto virsotņu skaitu (Georgyan, 2023). Kā uzsver iepriekš minētā avota autors, AlphaZero bija spējīgs sasniegt tik augstu meistarības līmeni, izmantojot tikai šaha spēles noteikumus un neskaitāmas pašspēles, no kurām mācīties. Tomēr, DeepMind izlēma AlphaZero programmatūru nepubliskot, kas radīja vēlmi un nepieciešamību izstrādāt alternatīvu, kas spētu atainot tā augsto veiktspējas līmeni (Klein, 2022). Iedvesmas vadīts, 2018. gadā Gary Linscott, amerikāņu datorzinātnieks, publicēja Leela Chess Zero (turpmāk Lc0) – uz neironu tīklu balstītu šaha programmatūru (Silver, 2018).

Lc0 izmantoja konvolūcijas neironu tīklu, jeb saīsināti no angļu valodas CNN - Convolution Neural Network, kas, neizmantojot nekādas cilvēka dotās zināšanas, izņemot tikai šaha spēles noteikumus un pašspēles no kurām mācīties, spēj ģenerēt stratēģiju, jeb politiku, kas apzīmē varbūtību katram gājienam un vērtību, kas norāda konkrētā gājiena, jeb pozīcijas novērtējumu (lczero, 2021). Iepriekš apskatītajā avotā tiek minēts arī tas, ka Lc0 izmanto PUCT algoritmu, kas tika aprakstīts 1.5. nodaļā, lai veiktu gājienu novērtēšanu.

Tikai divu mēnešu laikā Lc0 sasniedza Elo reitingu, kas bija robežās no 2600 līdz 2800, un tas prasīja 8,2 miljonus pašspēles no kurām mācīties (Silver, 2018).

Sākot no 2024. gada Lc0 ir sākuši izmantot transformatoru arhitektūru (lichess, 2024). Ar nebeidzamiem uzlabojumiem un aktīvu iesaistīšanos apmācības procesā, Lc0 šobrīd ir viens no labākajiem šaha dziņiem.

Ņemot vērā faktu, ka pašspēļu ģenerēšana ir sarežģīts process, kas prasa ievērojamus skaitļošanas resursus uz jaudīgu GPU, rodas jautājums, cik efektīvs varētu būt šis risinājums ierobežotu resursu situācijā. Šī iemesla dēļ ir svarīgi veikt dziļu izpēti literatūras avotos saistībā ar pašspēles apmācības metodi un praktiski eksperimentēt un pārbaudīt, vai ir iespējams šo metodi optimizēt, lai tā spētu efektīvi darboties arī uz ierobežotiem resursiem.

1.9. Algoritmu un metožu izvēle dziņa izstrādei uz ierobežotiem resursiem

Līdz šim tika apskatīti vairāki algoritmi, kas ir piemēroti sarežģītu uzdevumu risināšanai šaha spēles kontekstā. Katram no tiem ir atrodamas gan savas stiprās, gan vājās puses, jo īpaši situācijā, kad skaitļošanas resursi ir ierobežoti.

Minimax pārmeklēšanas algoritms, neskatoties uz tā precizitāti, pateicoties rūpīgai un secīgai virsotņu pārmeklēšanai spēles kokā, rada milzīgu skaitļošanas resursu patērtiņu. Tā lielākais mīnuss parādās pārmeklēto virsotņu skaita eksponenciālajā pieaugumā, kas ir novērojams gadījumā, kad spēles koka zarošanās faktors un koka dziļums ir liels, ko var novērot tieši šaha spēles situācijā. Šī iemesla dēļ Minimax algoritma izmantošana šajā situācijā būtu neefektīvs risinājums.

Alfa-Beta nogriešanas algoritms, kaut arī būdama optimizētāka versija Minimax algoritmam un būtiski samazina pārmeklēto virsotņu skaitu, tomēr joprojām patērē ievērojamu daudzumu resursu un turpina apskatīt tik lielu daudzumu virsotņu, kas padara to par ne pašu efektīvāko risinājumu šajā situācijā.

Montekarlo pārmeklēšanas algoritms, salīdzinot ar iepriekš apskatītajiem algoritmiem, uzskatāms par labāko risinājumu. Tā optimizētā versija PUCT, kas darbojas kopā ar neironu tīklu padara spēles koka pārmeklēšanu daudz efektīvāku, jau sākumā tiecoties uz virsotnēm, kas potenciāli varētu sniegt labus rezultātus, tādā veidā lieki netērējot resursus un laiku uz neveiksmīgu gājienu izpēti. Tomēr, šāda algoritma izmantošana patērētu daudz vairāk skaitļošanas resursus, salīdzinājumā ar situāciju, ja tiktu izmantota manuāli definēta heiristiskās novērtēšanas funkcija. Tomēr, jāņem vērā

fakts, ka šādas manuāli definētas funkcijas veidošana izrādās mazāk efektīva un sniedz sliktākus rezultātus dziņa apmācības procesā, nekā neironu tīkls, kas tika pierādīts un ir aprakstīts sīkāk 1.7. nodaļā. Šis noved pie situācijas, kas rada vēlmi pārbaudīt un salīdzināt abus variantus un to efektivitāti ierobežotu resursu situācijā – cik būtiska būs atšķirība dziņu veiktspējā un vai patērēto skaitļošanas resursu atšķirība būs tā uzlabojuma vērta.

2. APMĀCĪBAS METOŽU APSKATS

Apmācības metodes ir svarīga daļa no mākslīgā intelekta mūsdienās, kas ļauj sistēmām uzlabot savu sniegumu, atklājot apslēptas sakarības dotajos datos to apstrādes procesā. Apmācības metodes ir iespējams iedalīt vairākās kategorijās, galvenajām esot uzraudzītas (supervised), neuzraudzītas (unsupervised) un stimulētas (reinforcement) apmācības metodēm. Katra no metodēm piedāvā savas priekšrocības un trūkumus, attiecīgi ietekmējot to pielietojumu dažādās jomās (Sutton & Barto, 1998). Darba ietvaros tiks apskatītas neuzraudzītas un stimulētas apmācības metodes, rezultātā izveidojot divus dažādus šaha spēles aģentus un novērtējot to sniegumu.

2.1. Mašīnmācīšanās jēdziens

Mašīnmācīšanās (machine learning – ML) ir daļa no mākslīgā intelekta sfēras, kuras definīcija atšķiras atkarībā no apskatītā avota, katram sniedzot savu daļēji unikālu definīciju. Mašīnmācīšanās terminu 1959. gadā ieviesa Artūrs Samuels (Brown, 2021).

MIT Sloan raksts mašīnmācīšanos raksturo kā "mašīnas spēju atdarināt inteliģenta cilvēka uzvedību" (Brown, 2021).

Vienlaikus mašīnmācīšanās aizsācējs Artūrs Samuels mašīnmācīšanos definēja kā programmas mācīšanos no izspēlētajām spēlēm, balstoties uz savu pētījumu, kurā viņa izveidotā dambretes programma uzveica pašatzītu dambretes meistaru (IBM, n.d).

ISO/IEC 2382-28 1995. gada standarts mašīnmācīšanos raksturo kā "procesu, kurā funkcionāla vienība uzlabo savu veiktspēju, iegūstot jaunas zināšanas vai prasmes, vai arī pārkārtojot esošās zināšanas vai prasmes" (ISO/IEC, 1995). 2015. gada standartā definīcija ir nemainīga.

Vispārinātāka mašīnmācīšanās definīcija ir pieejama ISO tīmekļa vietnē, kur mašīnmācīšanās tiek definēta kā "mākslīgā intelekta veids, kas ļauj mašīnām mācīties no datiem bez nepieciešamības tās tieši programmēt" un tiek uzsvērta mašīnmācīšanās spēja pieņemt lēmumus vai veikt minējumus, kas balstās uz liela datu apjoma un ir pielīdzināmi inteliģentiem lēmumiem vai minējumiem (ISO, n.d). Šī darba ietvaros tiks izmantota ISO tīmekļa vietnes definīcija, jo tā precīzi raksturo datu nepieciešamību mašīnmācīšanās procesā, papildus uzsverot, ka netiek veiktas tiešas programmatūras izmaiņas, lai uzlabotu sistēmas veiktspēju.

2.2. Apmācības metodes divpersonu spēlēm ar pilnu informāciju

Šajā apakšnodaļā tiks plašāk aprakstītas apmācības metodes, kas var tikt pielietotas divpersonu spēlēm ar pilnu informāciju.

2.2.1. Uzraudzīta apmācība

Uzraudzītā apmācībā ir pieejami ieejas dati un tiem atbilstošās izejas vērtības, ļaujot aģentam "iemācīties" dažādas sakarības, kas varētu pastāvēt starp datiem (Jo, 2021). Piemērs – vēsturiskas šaha spēles, kur ir izskaitļoti pareizie gājieni. Pielietojot uzraudzīto apmācību, dati tiek sadalīti apmācības un testēšanas kopā, parasti 80% no datiem esot apmācības datiem un atlikušajiem 20% kalpojot kā testēšanas datiem, kas tiek izmantoti, lai pārbaudītu izveidotā modeļa veiktspēju.

Galvenie uzraudzītās apmācības veidi ir klasifikācija un regresija.
Klasifikāciju definē kā kādas iepriekš definēta kategorijas vai kategoriju piešķiršana (Jo, 2021, 22. lpp.). Klasifikācijas metodēs ietilpst tādi algoritmi kā lēmumu koki, naivais Beijess (*Bayes*), loģistiskā regresija un K tuvāko kaimiņu metode (Kartik, 2025).

Binārās klasifikācijas gadījumā spēles gājiens tiek klasificēts vienā no divām kategorijām (piemēram, "slikts" vai "labs") (Jo, 2021, 87. lpp.). Klasifikācijas metodes rezultāts ir atkarīgs no ieejas datiem, kuri varētu ietvert pašreizējo spēles stāvokli, novērtējuma funkcijas rezultātu un citu spēlei svarīgu informāciju.

Nebināras klasifikācijas gadījumā ir iespējams veikto gājienu klasificēt vairāk kā divās kategorijās, gājienam piešķirot vienu vai vairākas kategorijas (Jo, 2021, 89. lpp.). Piemēram, jebkuru gājienu ir iespējams klasificēt kā "lielisks", "labs", "neitrāls", "slikts" vai "katastrofisks". Nebināras klasifikācijas gadījumā tiek sniegta precīzāka informācija par gājiena kvalitāti, salīdzinot ar bināro klasifikāciju, tomēr arī šādā gadījumā ir novērojams diskrēts vērtību skaits, kas var būt nepietiekams precīzai spēles stāvokļa analīzei.

Klasifikācijas pielietošana iepriekšminētājos gājiena novērtēšanas gadījumos varētu ierobežot dziņa veiktspēju. Tas notiktu tādēļ, ka gājienu granularitāte samazinātos – klasifikācijas funkcijas iespējamās vērtības būtu ierobežotas, tādējādi sniedzot sliktāku priekšstatu par gājiena ietekmi uz spēli, salīdzinot ar skaitliskām novērtējuma funkcijām.

Klasifikācija ir labāk piemērota, ja nepieciešams atpazīt pašreizējo spēles stāvokli no, piemēram, sniegtā attēla, sniedzot iespēju noteikt visu spēles figūru atrašanās vietu uz laukuma.

Tomēr klasifikācijas stiprā puse ir atrodama spēļu analīzes kontekstā. Tā var sniegt, piemēram, nepieredzējušākam spēlētājam vieglāk saprotamu analīzi par spēles gaitā veikto gājienu kvalitāti. Šādas klasifikācijas efektivitāte ir atkarīga no izmantotā modeļa kvalitātes un izvēlēto kategoriju piemērotības konkrētajai spēlei.

Regresija tiek definēta kā izejas vērtības novērtēšanas process, atkarībā no ieejas vērtības. Atšķirībā no klasifikācijas, regresijas izejas vērtība ir nepārtraukta (Jo, 2021, 24. lpp.).

Regresijas izmantošana ar labi izstrādātu modeli sniegtu precīzu gājiena novērtējumu, kas, lai gan nebūtu tik viegli saprotams nepieredzējušākam spēlētājam, ļautu ievērojami precīzāk novērtēt un salīdzināt divus gājienus. Piemēram, ja klasifikācijas gadījumā divi gājieni varētu tikt klasificēti kā "labi", regresijas modelis varētu parādīt, ka viens no izvēlētajiem gājieniem ir gandrīz "neitrāls", bet otrs ir gandrīz "lielisks".

2.2.2. Neuzraudzīta apmācība

Neuzraudzītā apmācībā datiem netiek piešķirtas vēlamās izejas vērtības, liekot aģentam pašam atrast dažādas sakarības, kas pastāv dotajos datos (Gupta, 2025).

Galvenās neuzraudzītās apmācības pieeja ir klasterēšana, dimensiju samazināšana un asociatīvā likumu mācīšanās.

Klasterēšanā dati tiek iedalīti klasteros pēc to savstarpējās līdzības vienam ar otru. Daži no klasterēšanas algoritmiem ir hierarhiskā klasterēšana, k-vidējo klasterēšana, spektrālā klasterēšana u.c.

Hierarhiskā klasterēšanā dati tiek pakāpeniski grupēti vai pēc lielākās līdzības vienam ar otru (aglomeratīvā klasterēšana) vai atdalīti pēc lielākās nevienlīdzības (sadalošā klasterēšana), veidojot hierarhisku struktūru, ko var pielīdzināt kokam (Dey, 2025). Šādu klasterēšanas veidu ir iespējams vizualizēt ar dendrogrammas palīdzību, attēlojot to, kā vislīdzīgākie dati tiek pakāpeniski grupēti vai visatšķirīgākie dati tiek pakāpeniski atdalīti viens ar otru, klasterēšanas rezultātā iegūstot pilnībā apvienotu vai atdalītu datu kopu. Šādā dendrogrammā ir iespējams izvēlēties vēlāmo klasteru skaitu, kādā tiks iedalīti dati.

2.2.3. Stimulēta apmācība

Stimulētā apmācībā netiek doti spēļu dati, galvenokārt ļaujot aģentam pašam atklāt optimālos stratēģijas, gājienus vai darbības, pielietojot balvu sistēmu, atkarībā no tā, vai veiktā darbība tiek uzskatīta par pareizu vai nepareizu (Bajaj, 2025). Aģents, veicot gājienus vai noteiktas darbības, saņem pozitīvu balvu vai sodu, kas ļauj aģentam mainīt savu uzvedību, lai pēc iespējas maksimizētu kopējo balvu.

Stimulētā apmācībā ir svarīgi sasniegt pēc iespējas labāku balansu starp jaunu gājienu vai darbību izpēti un eksistējošo labu gājienu, darbību izmantošanu, rezultātā iegūstot pēc iespējas labāku modeli (Murel, Kavlakoglu, 2024).

Galvenās stimulētās apmācības sastāvdaļas ir politika, balvas signāls, vērtības funkcija un modelis (Bajaj, 2025).

Politika nosaka aģenta uzvedību, balstoties uz pašreizējo stāvokli (Bajaj, 2025). Politiku var pielīdzināt dzīvnieka reakcijai uz apkārtējo vidi. Piemēram, satraukta dzīvnieka politika būtu bēgt no iespējamiem draudiem, lai pasargātu sevi. Šādā gadījumā tiktu izvēlēta bēgšanas darbība, balstoties uz stāvokli (piemēram, konstatējot draudus) ar mērķi izvairīties no briesmām.

Balva nosaka galamērķi, piemēram, uzvaru spēlē. Aģenta mērķis ir pēc iespējas maksimizēt kopējo balvu (Murel, Kavlakoglu, 2024). Balvai apmācībā tiek izmantots pozitīvs un negatīvs pastiprinājums (Bajaj, 2025).

Pozitīvā pastiprinājumā aģentam tiek piešķirta balva par pareizas darbības vai gājiena veikšanu, novedot pie biežākas gājiena izmanošanas, kas, savukārt, var novest pie labākas aģenta veiktspējas, balvai pozitīvi ietekmējot aģentu.

Negatīvā pastripinājuma darbība atšķiras ar soda nepiešķiršanu, aģentam izvairoties no sliktas darbības vai gājiena, virzot aģentu tālāk no zināmiem sliktiem gājieniem, nodrošinot zināmu aģenta veiktspējas slieksni.

Vērtības funkcija atšķiras no balvas signāla, jo tiek pievērsta uzmanība ilgtermiņa labumam (Murel, Kavlakoglu, 2024). Stāvoklis tiek novērtēts pēc nākotnes balvu iespējām, vaodites pēc aģenta politikas. Vērtības funkcija ļauj izvērtēt, kādas darbības varētu sniegt vislabāko rezultātu pēc vairākām darbībām vai gājieniem, nodrošinot lielāku kopējo ieguvumu, salīdzinot ar tūlītēju gājienu novērtējumu.

Modelis ir neobligāta stimulētas apmācības aģenta daļa, kas ļauj tam paredzēt vides darbību uz iespējamām aģenta darbībām (Murel, Kavlakoglu, 2024). Ar modeļa

palīdzību aģents var veikt minējumus un novērtēt potenciālos rezultātus, neveicot darbību vai gājienu.

2.3. Metožu pielietojums šaha spēles kontekstā

Šahā, kā divpersonu spēlē ar pilnu informāciju, aģentu apmācībā galvenokārt tiek pielietota uzraudzītā un stimulētā apmācība, ko demonstrē tādi aģenti kā Stockfish, AlphaZero un Lc0 (lczero, 2021).

Uzraudzītā apmācība ietver aģentu apmācību ar jau izspēlētām partijām, kurām ir pieejams vai arī tiek piesaistīts pozīcijas novērtējums (piemēram, no Stockfish dziņa) vai spēles rezultāts. Datu kopa šādam apmācības veidam parasti sastāv no publiski pieejamām datubāzēm, kā, piemēram, lichess atvērtā datubāze, kurā ir atrodamas mēneša gaitā izspēlētās šaha partijas. Šo izspēlēto partiju piesaistītā papildu informācija (pozīciju novērtējums vai spēles gala rezultāts) palīdz aģentam "iemācīties", kuri gājieni ir optimāli un noved pie galvenā mērķa – uzvaras.

Atšķirībā no uzraudzītās apmācības, stimulētās apmācības gadījumā aģentam netiek dota datu kopa ar iepriekš izspēlētām partijām, kurām ir piesaistīts rezultāts vai individuālas pozīcijas novērtējums. Šaha aģents spēli iemācās, tam spēlējot lielu skaitu partiju pašam pret sevi.

Kā tika minēts 2.2. nodaļā, stimulētā apmācībā aģenta darbību vada tā politika, balva, vērtības funkcija un, retākos gadījumos, arī modelis. Balvas signāls sniedz aģentam atgriezenisko saiti par veiktā gājiena labumu.

Piemēram, šahā aģentam var piešķirt balvu, tam izvairoties no mata vai neoptimālas figūru apmaiņas (negatīvs pastiprinājums), vienlaikus piešķirot balvu par mata veikšanu pret pretinieku vai pretinieka figūras iegūšanu (pozitīvs apstiprinājums).

Darbā uzraudzītā apmācība tiks pielietota, lai izveidotu aģentu, kas mācās no eksistējošām spēlēm, izmantojot lichess spēļu datubāzi, ar mērķi pēc iespējas labāk apgūt šaha spēli, izmantojot ierobežotu apmācības laiku un izmantoto partiju skaitu. Vienlaikus stimulētā apmācība tiks izmantota, lai izstrādātu pašspēles aģentu, kuram būs pieeja tikai spēles noteikumiem un atļautajiem gājieniem pašreizējā spēles laukumā. Abu aģentu izstrādē tiks izmantota konvolucionālā neironu tīkla (CNN) arhitektūra.

2.4. Mākslīgo neironu tīklu definīcija

Mākslīgais neironu tīkls (turpmāk neironu tīkls) ir mašīnmācīšanās ietvaros pielietots modelis vai programma, kas spēj pieņemt lēmumus veidā, kas līdzinās cilvēka smadzeņu darbībai, izmantojot procesus, kas atdarina neironu kopdarbības principus (IBM, 2021). Papildus tam, avotā tiek minēts, ka neironu tīkla modeļi tiek izmantoti, lai atrastu dažādas sakarības, izsvērtu dažādas iespējas un nonāktu pie secinājumiem.

Mākslīgā neironu tīkla galvenā sastāvdaļa ir mākslīgais neirons, kuri, veidojot savienojumus ar citiem neironiem, rezultātā izveido neironu tīklu (IBM, 2021).

Katrs neironu tīkls sastāv no ievades slāņa, viena vai vairākiem slēptajiem slāņiem un izvades slāņa (IBM, 2021). Neironu tīklus mēdz arī iedalīt dziļos (angļu val. *deep neural networks)* un seklos (angļu val. *shallow neural networks)* neironu tīklos, kur seklie tīkli parasti sevī ietver vienu slēpto slāni, bet dziļie tīkli – vairākus slēptos slāņus (Cloudflare, n.d.).

2.5. Neironu tīklu veidi

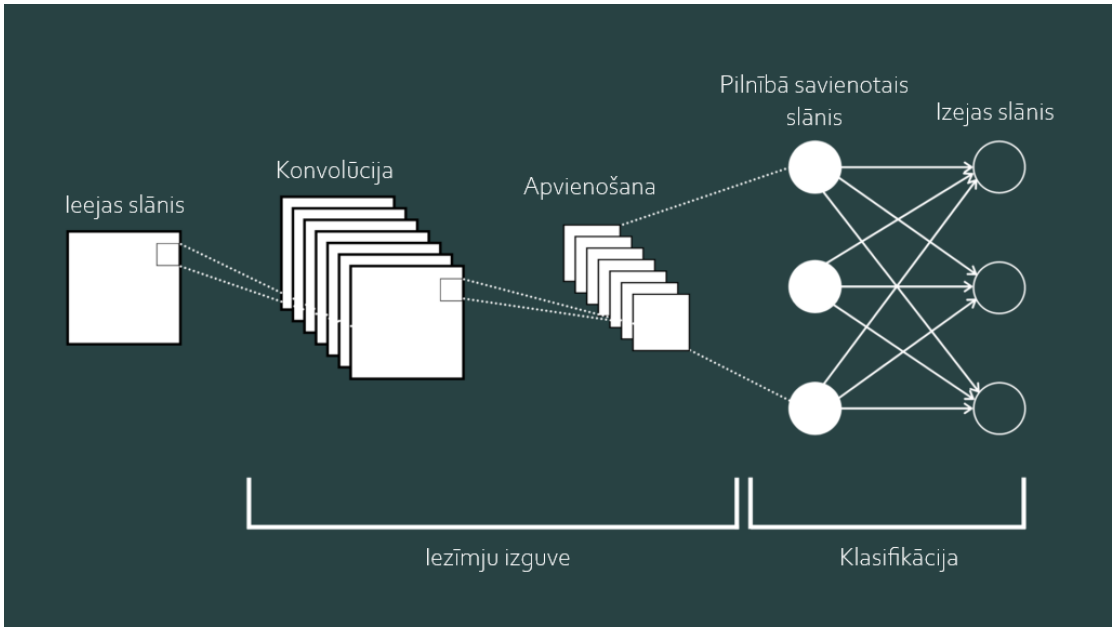
Mākslīgos neironu tīklus ir iespējams iedalīt daudzos veidos (Cloudflare, n.d.). Daži no plašāk pielietojamiem neironu tīklu veidiem ir vienslāņa un daudzslāņu perceptroni, Konvolucionālie neironu tīkli, rekurentie neironu tīkli, transformatoru modeļi (angļu val. *Transformer model*), un, šaha gadījumā arī efektīvi atjaunināmi neironu tīkli (angļu val. *Efficiently Updatable Neural Networks*).

Vienslāņa un daudzslāņu perceptronu tīkli atšķiras tikai ar to slēpto slāņu skaitu – vienslāņa perceptrona tīkls sastāv tikai no ieejas un izejas slāņiem, kamēr daudzslāņu perceptrons sevī iekļauj vienu vai vairākus slēptos slāņus (Cloudflare, n.d.).

2.5.1. Konvolucionālais neironu tīkls

Konvolucionālais neironu tīkls (CNN) ir specializēta neironu tīkla paveids, kas ir piemērots attēlu vai dažādu režģveida datu apstrādei (Mishra, 2025). CNN tīkli bieži tiek izmantoti objektu atpazīšanai attēlos vai video, runas atpazīšanai un klasifikācijas uzdevumos – piemēram, vai dotajā attēlā ir redzams tomāts vai ābols

Vizuāls CNN arhitektūras pārskats ir pieejams 2.2. attēlā.



2.2. att. Tipiska CNN tīkla arhitektūra (modificēts no (The Click Reader, n.d.))

CNN tīkli sastāv no trīs galvenajiem slāņu veidiem:

1. Konvolūcijas slānis,
2. Apvienošanas slānis (angļu val. *Pooling layer*),
3. Pilnsaistes slānis (FC).

Konvolūcijas slānis, izmantojot filtrus (nelielas svaru matricas), izgūst dažādas iezīmes no dotajiem ieejas datiem. Ieejas dati tiek pakāpeniski apstrādāti, skatoties uz nelieliem apgabaliem, kurus sauc par uztveres laukiem (Gillis, 2024).

Apvienošanas slānis seko konvolūcijas slānim. Apvienošanas slāņa galvenais uzdevums ir veikt dimensiju skaita samazināšanu, tādējādi uzlabojot tīkla efektivitāti un samazinot pārmerīgas pielāgošanas – ieejas datu iegaumēšanas – risku (Gillis, 2024). Dimensiju skaita samazināšana noved pie daļēja informācijas zuduma, ko atsver iepriekšminētā pārmērīgās pielāgošanās riska samazinājums un efektivitātes pieaugums.

Pilnsaistes slānī katrs neirons ir savienots ar visiem iepriekšējā slāņa neironiem (Gillis, 2024). Šis slānis veic klasifikāciju vai regresiju, balstoties uz tīklā identificētajām iezīmēm. Piemēram, attēlu klasifikācijā FC slānis varētu izvadīt varbūtību, ka dotajā attēlā ir redzams kaķis ar varbūtību 0,95 un zilonis ar varbūtību 0,13.

Papildus iepriekšminētajiem slāņiem, nereti CNN tīklos ir sastopami arī aktivācijas slāņi (Mishra, 2025). Viens no aktivācijas funkcijas piemēriem ir

iepriekšminētā ReLu aktivācijas funkcija, kuras izmantošana CNN tīklos sniedz tīklam iespēju iemācīties sarežģītākas sakarības, ieviešot nelinearitāti.

2.5.2. Transformatoru modeļi

Transformatoru modeļi ir neironu tīklu arhitektūra, kas ir piemērota secīgu datu apstrādei un bieži asociēta ar lielajiem valodu modeļiem (LLM), arhitektūrai sākotnēji tiekot ieviestai kā rekurento neironu tīklu evolūcija (Stryker, Bergmann, 2025). Transformatoru modelis darbojas, sekojot līdzi sakarībām secīgos datos un iemācoties to kontekstu, izmantojot matemātiskus paņēmienus, kurus sauc par uzmanību vai pašuzmanību (Merritt, 2022). Pašuzmanību var raksturot kā algoritmus, kas nosaka, kuras ir svarīgākās datu virknes daļas, kurām aģentam vajadzētu "pievērst uzmanību" dotajā laika posmā (Stryker, Bergmann, 2025). Šī spēja modelēt datu sakarības un uztvert valodas nianses padara transformatoru modeļus par spējīgu un plaši pielietotu rīku lielos valodu modeļos.

Papildus lielo valodu modeļiem, transformatoru modeļus pielieto attēlu izšķirtspējas uzlabošanā. Piemēram, NVIDIA izstrādātā DLSS (Deep Learning Super Sampling) tehnoloģija izmanto transformatoru arhitektūru, lai reāllaikā mērogotu kadrus no zemākas sākotnējās izšķirtspējas uz augstāku mērķa izšķirtspēju, tādējādi ļaujot sasniegt augstāku kadru skaitu sekundē, saglabājot tikpat augstu vai attēla kvalitāti.

Transformatoru arhitektūra, daļēji balstīta uz relāciju datubāu principiem, katrai datu virknes daļai ģenerē trīs sekojošos vektorus, ar kuru palīdzību tiek aprēķināti uzmanības svari (Stryker, Bergmann, 2025):

1. Vaicājuma (Q) vektors, kas atspoguļo informāciju, ko konkrētā tekstvienība "meklē" - kādu iespaidu citas tekstvienības varētu uz to atstāt.

2. Atslēgas (K) vektori pārstāv informāciju, kas atrodas noteiktajā tekstvienībā. Sastatījums starp vaicājuma un atslēgas vektoriem tiek izmantots, lai aprēķinātu uzmanības svarus.

3. Vērtības (V) vektors, kas "atgriež" informāciju no tekstvienības, tai tiekot mērogotai atkarībā no attiecīgā uzmanības svara. Tādējādi tekstvienības, kuru atslēgas vektori labāk saskan ar vaicājuma vektoru, saņem lielāku uzmanības svaru un to sniegtās informācijas ieguldījums ir spēcīgāks.

Pēc vaicājuma un atslēgas vektoru ģenerēšanas, lai noteiktu to savstarpējo atbilstību, tiek iegūti to skalārie reizinājumi, reizinot vaicājumu matricu ar transponētu atslēgas matricu, tādējādi iegūstot rezultātu matricu (Ferrer, 2024). Šī matrica parāda, cik lielu uzmanību katrai tekstvienībai būtu jāpievērš citai tekstvienībai. Augstāks iegūtais rezultāts norāda uz lielāku uzmanību starp tekstvienībām. Piemēram, šaha aģenta gadījumā, ja b1 lauciņš spēles sākumā veic uzmanības noteikšanu, tam varētu rasties augsts saskaņas punkts ar c3 vai d2 lauciņu, jo tie ir visbiežāk veiktie zirga gājieni.

Pēc rezultātu matricas iegūšanas, tā tiek mērogota. Rezultātu matricas saskaņas punkti parasti tiek mērogoti, dalot tos ar atslēgas un vaicājumu vektoru kvadrātsakni, pēc tam pielietojot softmax funkciju – ierobežojot vērtības robežās no 0 līdz 1 (Ferrer, 2024). Rezultātā tiek iegūti uzmanības svari, kuru kopējā summa ir 1.

Lai modelis spētu vienlaicīgi uztvert dažādas tekstvienību saistības, transformatori izmanto vairākgalvu uzmanību (angļu val. *multi-headed attention*). Vairākgalvu uzmanībā vaicājuma, atslēgas un vērtības vektori tiek lineāri pārveidoti, atkarībā no galvu skaita, katrai pārveidošanai paralēli izpildot uzmanības algoritmus un beigās atkalapvienojot rezultātus (Ferrer, 2024).

Papildus transformatoru tīkli sastāv arī no vienvirziena tīkla, kas tiek pielietota pēc uzmanības slāņa. Vienvirziena tīkls parasti sastāv no diviem lineāriem slāņiem ar ReLu aktivācijas funkciju starp slāņiem (Ferrer, 2024).

Beigu rezultātā no transformatora tiek iegūta katrai ievades teksvienībai atbilstošs, apstrādāts vektors ar dažādām iezīmēm, kas ir piemērots tālākai izmantošanai.

Pēdējos gados ir novērojama tendence transformatoru modeļiem aizstāt CNN un rekurentos neironu tīklus. Šo tendenci ilustrē fakts, ka laika posmā no 2020. līdz 2022. gadam 70% no arXiv arhīvā publicētajiem rakstiem mākslīgā intelekta jomā piemin transformatoru arhitektūru, kamēr 2017. gadā rekurentie un CNN tīkli bija populārākās neironu tīklu arhitektūras (Merritt, 2022). DLSS ir viens no piemēriem, kur, sākot ar ceturto versiju, tika pāriets no CNN uz transformatoru arhitektūru, rezultātā sniedzot uzlabotu attēlu kvalitāti (Lin, Burnes, 2025).

2.5.3. Efektīvi atjaunināmais neironu tīkls

Efektīvi atjaunināmais neironu tīkls jeb NNUE ir sākotnēji Šogi (angļu val. *Shogi*) veidota neironu tīklu arhitektūra, kas izceļas ar savu nelielo tīkla struktūru un iespēju atjaunināt tikai daļu no tīkla, bez nepieciešamības atjaunināt visu tīklu (Official-stockfish, 2025).

Tieši šis ātrums, ko nodrošina nelielā izmēra neironu tīkls, ir padarījis NNUE par pievilcīgu risinājumu tādiem šaha dziņiem kā Stockfish, jo tas ļauj darbināt dzini, izmantojot tikai procesoru, kā arī nelielais tīkla izmērs iespējo apskatīt daudz vairāk pozīciju vienā sekundē nekā dziļāki tīkli, kas neironu tīka realizācijai nepieciešamo aprēķinu veikšanai izmanto videokarti.

2.6. Neironu tīkli šaha spēles kontekstā

Šahā plaši tiek pielietotas divas galvenās neironu tīklu arhitektūras – transformatoru un NNUE – kaut arī pēdējos gados pārsvaru ir ieguvis NNUE, uz ko norāda TCEC Cup 14 (27. sezonas noslēdzošā kausa izcīņas) rezultāti, kur daudzi no spēcīgākajiem vai jaunajiem dziņiem balstās uz NNUE, finālā satiekoties Stockfish un Lc0 dziņiem (TCEC, n.d.).

Šis NNUE pārsvars atspoguļo tā spēju ātri un precīzi novērtēt pozīcijas tikai ar procesoru. Tomēr dziļāki tīkli, kopā ar MCTS, kā to demonstrēja AlphaZero un turpina attīstīt tādi dziņi kā Lc0 un Ceres, joprojām ir konkurētspējīgi ar jaudīgākajiem pašreiz esošajiem NNUE dzinējiem, kā redzams 2.1. tabulā, ar turnīra 4 labākajiem dziņiem un to izmantotajām neironu tīklu arhitektūrām.

2.1. tabula

TCEC Cup 14 rezultāti (modificēts no (TCEC, n.d.))

Iegūtā vieta	Dziņa nosaukums	Neironu tīklu arhitektūra
1.	Stockfish	NNUE
2.	Lc0	Transformatoru modelis
3.	Ceres	Transformatoru modelis
4.	Berserk	NNUE

Darbā apmācības metožu salīdzināšanai tiek pielietota CNN tīkla arhitektūra. Lai arī pēdējos gados CNN arhitektūra, skatoties uz pašlaik spēcīgākajiem šaha dziņiem, ir tikusi izkonkurēta, pārsvaru iegūstot NNUE un transformatoru modeļiem, Šī darba mērķis ir veikt metožu salīdzinājumu ar ierobežotiem skaitļošanas resursiem. CNN ir plaši pētīta un labi paskaidrota arhitektūra šaha spēles kontekstā, padarot to par pietiekami spējīgu izvēli darba mērķa sasniegšanai.

NNUE arhitektūra, lai gan īpaši efektīva ātriem gājiena novērtējumiem, pielieto ļoti specializētu spēles stāvokļa kodēšanu, ar nepieciešamību pēc zemāka līmeņa optimizācijām. Kaut arī šī arhitektūra ir pietiekami aprakstīta Stockfish dziņa gadījumā, tās pielietošana rezultētos daļējā Stockfish dziņa arhitektūra replicēšanā, novirzoties no galvenā mērķa – apmācības metožu salīdzināšanas – uz specifiskas dziņa arhitektūras atveidošanu, un varētu radīt grūtāk izprotamas sekas aģenta uzvedībā, sarežģījot apmācības metožu salīdzinājumu.

3. AĢENTA REALIZĀCIJA

Darbā izveidotie modeļi tiek apmācīti un palaisti uz šādas sistēmas:

- Procesors: AMD Ryzen 7 5800X3D,
- Videokarte: Nvidia Geforce RTX 5080 – 16GB GDDR7,
- RAM: 32GB 4x8GB 3200Mhz CL16
- SSD: 1TB PCIe 3.0 x4 Opērētājsistēmai un WSL, 4TB PCIe 4.0 x4 spēļu glabāšanai un nolasīšanai

Darbā izmantotās programmatūras versijas:

- Operētājsistēma: WSL2 2.4.11.0 uz Windows 11 64-bit 10.0.26100
- Python: 3.12.7
- PyTorch: 2.7.0+cu128

Darbs tiek veidots uz Microsoft Windows 11 operētājsistēmas ar WSL2 (Windows Subsystem for Linux). WSL primārā funkcija ir iespējot Linux vides pielietošanu bez nepieciešamības izveidot atsevišķu virtuālo mašīnu, kā arī dot tiešu pieeju sistēmas skaitļošanas resursiem.

PyTorch Linux versijai ir augstāka veiktspēja nekā pielīdzināmai Windows versijai, tādējādi paātrinot apmācības laiku. Papildus tam, ir nepieciešama atbilstoša PyTorch versija ar nepieciešamo CUDA (Compute Unified Device Architecture) lietojumprogrammas saskarnes (API) versiju 12.8 vai jaunāku, kas iespējo atbalstu NVIDIA Blackwell videokartēm ar kuru tiek veikta modeļa apmācība.

PyTorch tiek izmantots kopā ar PyTorch Lightning bibliotēkas palīdzību, kas atvieglo modeļa apmācības uzstādīšanu. Papildus tiek arī izmantotas “chess”, ”h5py”, “hdf5plugin” un “tqdm” bibliotēkas atvieglotai spēļu nolasīšanai, efektīvākai spēļu saspiešanai un apmācības izpildes progresa vizualizācijai.

3.1. Gājienu un spēles stāvokļu kodēšana neironu tīklam

Lai neironu tīkls spētu uztvert un attiecīgi apstrādāt šaha spēles stāvokļus, ir nepieciešams tos pārvērst skaitliskos formātos. Viens spēles stāvoklis tiek iekodēts 18 dažādās spēles laukuma 8x8 izmēra plaknēs, kur katra no tām attēlo dažādus spēles stāvokļa aspektus.

Pirmās 12 plaknes ataino katra spēlētāja unikālo figūru (bandinieks, zirgs, laidnis, tornis, dāma, kralis) novietojumu uz spēles lauciņa, kur ar 1 tiek atzīmēta konkrētā kauliņa pozīcija uz laukuma, svaukārt ar 0 atzīmē laukumus, kuros šis kauliņš neatrodas.

Nākamā, 13. plakne ataino, kurai krāsai piemīt pašreizējais gājiens, kur ar 1 tiek atzīmēts balto figūru gājiens, savukārt ar 0 – melno figūru gājiena kārta.

Turpmākās 4 plaknes attēlo rokādes iespējas, kur ar 1 tiek atzīmēta rokādes iespēja uz karaļa, vai karalienes pusi attiecīgajai krāsai.

Pēdējā plaknē tiek atainots, kāds ir materiālais balanss, jeb kāda ir materiālā figūru vērtība katrai krāsai, kur detalizētāk katras figūras vērtība tika aprakstīta iepriekš, 1.3 nodaļā. Spēlētājs ar lielāku figūru vērtību tiek uzskatīts par vadībā esošu, jeb lielākām priekšrocībām. Materiālais balanss tiek skaitliski attēlots un mērogots robežās no 0-255, kur 0 nozīmē melno figūru pilnīgu pārākumu pār baltajām, 255 atainos balto figūru pilnīgu pārākumu, savukārt 128 nozīmēs balansētu stāvokli, kur abiem spēlētājiem ir vienāda materiālā vērtība. Materiālais balanss tiek aprēķināts no balto figūru perspektīvas:

```
total_material = white_material + black_material
if total_material > 0:
    white_ratio_scaled = int((white_material / total_material) * 255.0)
    planes[17].fill(max(0, min(255, white_ratio_scaled)))
else:
    planes[17].fill(128)
```

Iemesls, kāpēc tika izvēlēts attēlot šo vērtību 0-255 robežās ir skaidrojams ar to, ka jebkurš skaitlis šajās robežās ietilpst vienā baitā, kas attiecīgi nodrošina efektīvu atmiņas izmantošanu un ietaupīšanu, kā arī nodrošina vienkāršu turpmāko normalizāciju neironu tīklam.

Gājienu kodēšanas procesā, kur gājiens, kas tiek attēlots šādi, piemēram – e2e4, tiek pārveidots par unikālu skaitlisku indeksu robežās no 0 līdz 1967. Uz tukša spēles laukuma katra kvadrāta tiek ģenerēti pseido-legāli gājieni katrai unikālai šaha figūrai balstoties uz to kustības loģiku, kur, piemēram, laidnis kustās pa diagonāli, tornis – taisni, zirgs – "L" formā utt. Šie gājieni, kas tiek ģenerēti ir pseido-legāli, jo tie neņem vērā gājienu legalitāti atkarībā no konkrētā spēles stāvokļa, piemēram, vai lauciņš uz kuru figūra tiek pārvietota ir aizņemts. Iemesls tam ir saistīts ar to, ka kodēšanas procesā

tas nav nepieciešams – gājienu legalitāte, neatkarīgi no spēles stāvokļa, tiek pārbaudīta un nelegāli gājieni netiek pieņemti citā koda daļā, pašā spēles laikā.

Pēc tam, kad visi gājieni ir uzģenerēti, tie tiek saglabāti un katram no tiem piešķirts unikāls indekss. Lai saprastu to, kā notiek gājienu indeksēšana, sākumā jāaaplūko šaha spēles laukums. Šis laukums satstāv no 64 rūtiņām, kas tiek numurētas no 0 līdz 63 un skaitītas no kreisā apakšējā stūra un pa labi, turpinot uz augšu. Gājiens tiek apzīmēts ar rūtiņas numuru, no kura tas sāk gājienu un ar to numuru, uz kuru rūtiņu tas parvietojas. Papildus tam tiek ņemti vērā bandinieku paaugstināšanas, jeb pārvēršanās gadījumi par kādu no citām figūrām, kad tie nonāk spēles laukuma pirmajā vai astotajā rindā, atkarībā no figūru krāsas. Atkarībā no tā, uz kādu figūru notiek paaugstināšana, katrai no tām tiek iedalīts unikāls numurs, kur karalienei būs 5, tornim – 4, laidnim – 3, zirgam – 2, savukārt, ja paaugstināšana nonetiek, tad liek 0. Visi šie gājieni tiek saglabāti attiecīgajā formātā un secīgi sakārtoti, lai pēc tam tiem varētu piešķirt indeksus:

```
self.all_possible_moves = sorted(list(move_set), key=lambda m: (m.from_square,
m.to_square, m.promotion or 0))
        self.move_to_index = {move: idx for idx, move in
enumerate(self.all_possible_moves)}
```

3.2. Neironu tīkls

Šaha aģentu apmācībai tiek pielietots konvolūcijas neironu tīkls, kura mērķis ir no dotā šaha spēles stāvokļa noteikt divas lietas – politiku (angļu val. *policy*), kas norāda uz labākajiem gājieniem, ko ir iespējams veikt no dotās pozīcijas un vērtību, kas ataino dotā gājiena efektivitāti, jeb pozīcijas iznākumu, kas pasaka, vai spēlētājs šobrīd dodas pretī uzvarai, vai tomēr zaudējumam. Abi izstrādātie šaha aģenti izmanto vienotu neironu tīklu.

Pirmais konvolūcijas neironu tīkla slānis sastāv no 128 kanāliem, jeb filtreim. Katrs no šiem filtreim ir 3x3 izmērā un vienlaicīgi apstrādā visas 18 8x8 izmēra plaknes, kurās ir iekodēts šaha spēles stāvoklis (kā aprakstīts 3.1. nodaļā). Katram no 128 filtriem tiek piešķirti unikāli svari, kas palīdz atpazīt un uztvert dažādus, iespējams, nozīmīgus rakstus, jeb figūru izkārtojumu uz spēles lauciņa. Svari laika gaitā tiek pielāgoti, jeb apmācīti, atkarībā no tā, cik nozīmīgus rakstus filtri ir atklājuši. Katrs no šiem filtriem, paralēli darbojoties, rezultātā izdod vienu pazīmju karti (angļu val.

feature map) 8x8 izmērā. Šajās 128 kartēs ir izceltas pazīmes, jeb lauciņi uz spēles galda, kuros šie filtri, no dotā spēles stāvokļa, ir novērojuši kādu būtisku ainu.

Nākamajā solī uz šīm kartēm tiek pielietota ReLU aktivācijas funkcija, kas vērtības, kuras ir negatīvas pārvērš par 0, savukārt pārējās vērtības atstāj tādās pašas. Iemesls šim ir saistīts ar nelineraitātes ieviešanu datos, kas turpmākajos soļos būs nepieciešams, lai neironu tīkls spētu atrast un iemācīties sarežģītas saistības starp datiem un izprastu šaha spēles pozīcijas daudz efektīvāk.

Tālāk šīs kartes tiek nosūtītas uz 20 reziduālajiem (angļu val. *residual*) blokiem, kas apstrādā šo informāciju daudz sīkāk, meklējot sarežģītākas saistības starp ainām dotajā spēles pozīcijā. Reziduālo bloku skaits ir nozīmīgs arhitektūras parametrs, kura optimālo vērtību ir nepieciešams piemeklēt. Katrā no reziduālajiem blokiem notiek sekojošas darbības:

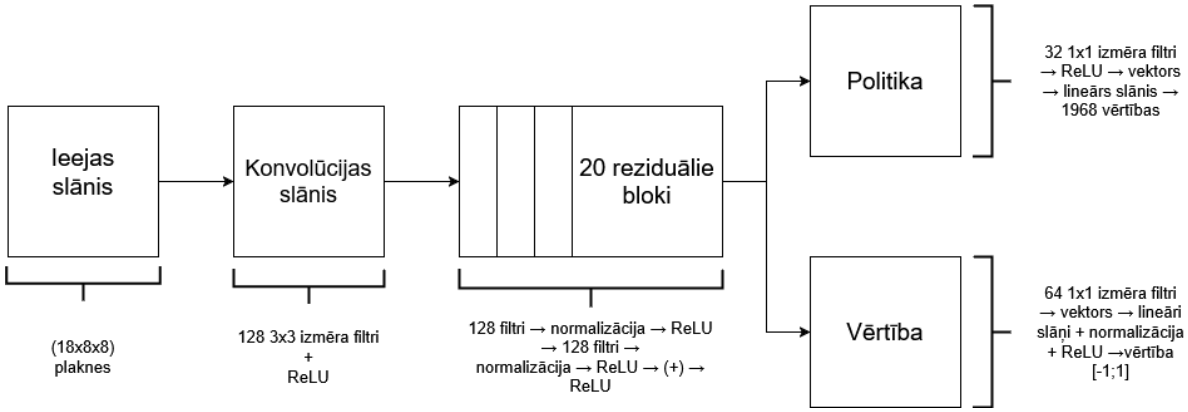
1) Saņemtās pazīmju kartes tiek kopētas un saglabātas, kas būs nepieciešamas vēlākām darbībām;
2) Uz šīm kartēm, tāpat kā pirmajā slānī tiek pielietoti 128 filtri, kas meklē būtiskas ainas un pazīmes dotajā šaha spēles stāvoklī, rezultātā dodot jaunas 128 kartes ar jaunām izceltām, novērotām ainām;
3) Iegūtie jaunie dati tiek normalizēti un uz tiem tiek pielietora ReLU funkcija;
4) Tālāk tiek atkal veikta jaunu filtru pielietošana un kārtējo jauno pazīmju karšu iegūšana, kuru rezultāti pēc tam atkal tiek normalizēti;
5) Šajā solī jaunās pazīmju kartes tiek apvienotas ar veco kopju, kas tika saglabāta pirmajā solī, ar mērķi nepazaudēt sākotnējos datus un atrastās pazīmes visu šo pārmaiņu rezultātā. Šie apvienotie dati tiek izlaisti cauri ReLU funkcijai un ir gatavi doties uz nākamo reziduālo bloku.

Politika – vislabāko gājienu noteikšana no dotā spēles stāvokļa – notiek, vispirms pielietojot 32 1x1 izmēra filtrus. Filtru izvadei tiek piemērota ReLu aktivācijas funkcija, veidojot jaunu pazīmju karšu kopu. Pēc tam pazīmes tiek saplacinātas vienā vektorā, pēc kā tas tiek padots lineārā pilnsaistes slānī, kas pārveido vektoru, lai iegūtu skaitlisku vērtējumu katram no 1968 iespējamajiem gājieniem, atbilstoši 3.1. nodaļā aprakstītajai gājienu kodēšanai. Politikā augstāks skaitlisks vērtējums norāda uz efektīvāku gājienu.

Vērtība nosaka pašreizējās pozīcijas efektivitāti. Sākotnēji tiek pielietoti 64 1x1 filtri, kam seko ReLu aktivācijas funkcija. Iegūtās pazīmju kartes pēc tam tiek pārveidotas par vektoru. Iegūtais vektors tiek apstrādāts ar vairākiem pilnsaistes

lineāriem slāņiem ar sekojošu pašnormalizāciju (angļu val. batch normalization) un ReLu aktivācijas funkciju. Savienotie slāņi pakāpeniski samazina dimensiju skaitu līdz 256 neironiem un, visbeidzot, pēdējā slānī tiek izvadīta skaitliska vērtība no -1 līdz 1, kas tiek iegūta, izmantojot hiperboliskā tangensa aktivācijas funkciju. Skaitliska vērtība 1 ir pielīdzināma pārliecinošai uzvarai, bet -1 ir pielīdzināma pārliecinošam zaudējumam. Lineāro slāņu iekļaušana ir vēl viens nozīmīgs, piemeklējams arhitektūras elements.

Vizuāls konvolūcijas neironu tīkla arhitektūras attēlojums ir redzams 3.1. attēlā.



3.1. att. Konvolucionālā neironu tīkla arhitektūra

Parametru skaits, ar kādiem neironu tīkls darbojas sniedz vairākus būtiskus secinājumus par tā veiktspēju un piemērotību dotajam uzdevumam. Piemēram, modelis ar pārāk mazu parametru skaitu var nespēt pilnībā izprast šaha spēles būtību un atrast kopsakarības starp gājieniem un pozīcijām, savukārt tīkls ar pārāk lielu parametru skaitu var pārāk pielāgoties apmācības datu kopai un nespēt pilnvērtīgi darboties uz jauniem, vēl neredzētiem datiem. Tāpat parametru skaits nosaka to, cik daudz skaitļošanas resursus tas patērēs un cik laikietilpīgs būs šis apmācības process.

Lai detalizēti novērtētu un rezultātā varētu nonākt pie secinājumiem saistībā ar modeļa veiktspēju, tiek aprēķināts kopējais neironu tīkla parametru skaits ar kuriem tas darbojas, un to sadalījums pa slāņiem ir attēlots 3.1. tabulā.

3.1. tabula

Konvolūcijas neironu tīkla parametru skaita sadalījums pa slāņiem

Neironu tīkla slānis	Parametri
Konvolūcijas slānis	20864
20 reziduālie bloki	5908480
Politika	4036560
Vērtība	2239041
Kopā	**12204945**

3.3. MCTS pārmeklēšanas algoritms ar PUCT

Lai nodrošinātu to, ka šaha dzinis veic dziļāk analizētus lēmumus nākamā gājiena izvēles procesā, tiek izveidots MCTS pārmeklēšanas algoritms, kas, izmantojot neironu tīkla dotos rezultātus, veic spēles koka pārmeklēšanu un gājienu simulēšanu, rezultātā dodot precīzāku prognozi par gājienu efektivitāti.

Algoritma darbība sākas ar to, ka tiek saņemts esošais šaha spēles stāvoklis un iekodēts attiecīgajā formātā, kas pēc tam tiek nodots neironu tīklam. Neironu tīkls kā rezultātu atdod atpakaļ algoritmam politiku, jeb nākamos potenciālos gājienus un vērtību šim konkrētajam gājienam. Šīs izejas vērtības MCTS algoritms normalizē robežās no 0 līdz 1 izmantojot Softmax funkciju. Šīs normalizētās vērtības ataino gājienu varbūtību, kur gājiens, kas apzīmēts ar skaitli, kas tuvāks 1 robežai būs daudzsološāks un efektīvāks, nekā tie gājieni, kuru varbūtība būs tuvāk nullei.

Nākamajā solī agloritms pielieto PUCT formulu (1.2) ar kuru izvēlas nākamo gājienu spēles kokā kuru apskatīt. Šī izvēle svārstās starp priekšrokas došanu tiem gājieniem, kurus neironu tīkls ir novērtējis kā daudzsološus un starp tiem, kuri vēl nav bijuši tik bieži apmeklēti, taču potenciāli varētu dot labākus rezultātus, ja izpētīti dziļāk. Atkarībā no tā, kādā stadijā spēle atrodas – sākumā, vidū, vai beigās - ir iespējams mainīt un ekspeirimentēt ar šo hiperparametru, kas nosaka cik lielu prioritāti likt neizpētītiem gājieniem.

Momentā, kad algoritms sasniedz jaunu gājienu, jeb spēles koka virsotni, kurai nav izpētīti pēcteči, tas iegūst neironu tīkla novērtējumu un uzsāk atpakaļizplatīšanas procesu atjauninot visu iepriekšējo gājienu apmeklējuma skaitu, kā arī novērtējumu, kas noveda līdz šim stāvoklim. Gājienu novērtējuma atjaununāšana notiek pēc

formulas, kurā, tiek izmanotas iegūtās jaunās un iepriekšējās spēles stāvokļu vērtības, kā arī ņemts vērā gājiena apmeklējumu skaits:

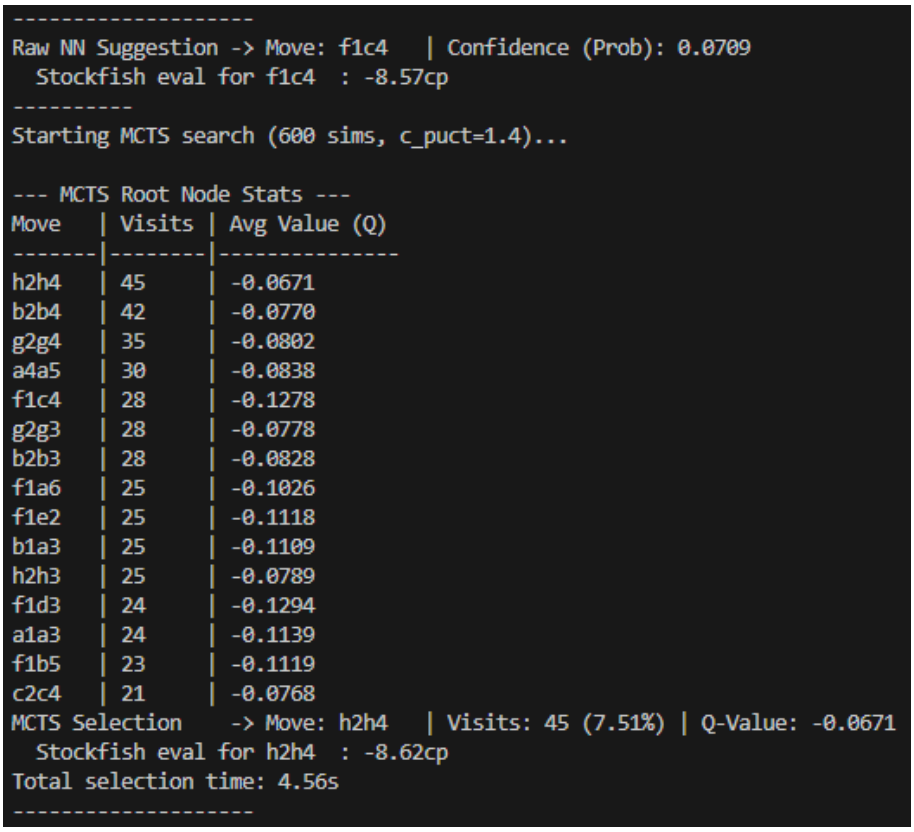
```
def backpropagate(self, path: list[str], value: float):
    current_value = -value
    for move_key in reversed(path):
      self.N[move_key] += 1
      self.Q[move_key] += (current_value - self.Q[move_key]) / self.N[move_key]
      current_value *= -1
```

Jāņem vērā arī tas, ka katrā spēles koka līmenī, kas tiek apskatīts, mainās gājiena perspektīva, kas nozīmē, ka ja esošais gājiens ir novērtēts ar pozitīvu vērtību, tad pretiniekam tā būs negatīva.

Tā šis process turpinās līdz noteiktam simulāciju skaitam, kur beigās algoritms izvēlas labāko gājienu, ko izspēlēt balstoties uz to, kura virsotne tikai apskatīta visvairāk.

3.3.1. Aģentu veiktspējas testēšana ar MCTS

Lai pārliecinātos, vai MCTS algoritma ieviešana uzlabo aģenta veikstspēju spēles laikā, tika ievests eksperiments kurā vienam un tam pašam aģentam liek spēlēt pašam pert sevi, tikai ar nosacījumu, ka no vienas puses tas spēlēs izmantojot tikai neironu tīklu, taču no otras puses tas spēlēs izmantojot neironu tīklu ar MCTS algoritmu. Lai gūtu labāku ieskatu par to, kā MCTS algoritms darbojas, tad spēles laikā, kad aģents, kurš izmanto šo algoritmu veic gājienu, tiek izprintēta tabula, kurā ir atainoti top 15 gājieni, kurus algoritms ir apskatījis un sakārtoti dilstošā secībā, kas ir redzams 3.1 attēlā. Papildus tam, ir atainots arī gājiens, kuru neironu tīkls iesaka kā labāko, ko pēc tam var salīdzināt ar to gājienu, ko algoritms ir izvēlējies pēc simulāciju veikšanas. Šiem abiem gājieniem tiek noteikta arī Stockfish novērtējums, kas palīdz izprast, cik efektīvs šis gājiens varētu būt. Šāda tipa vizuāls atainojums palīdz izprast to, cik bieži MCTS algoritms izvēlas sekot neironu tīkla ieteiktajiem gājieniem un cik bieži tas izvēlas izpētīt citus gājienus un izvēlēties potenciāli veiksmīgākus. 3.1 attēla piemērā ir redzama situācija, kurā MCTS algoritms neizvēlas sekot neironu tīkla ieteikumam (f1c4) un rezultātā izspēlē pavisam savādāku gājienu (h2h4), kas pēc Stockfish evaluācijas izrādās nedaudz labāks. Stockfish novērtē gājienus skatoties no balto figūru perspektīvas, kas nozīmē, ja vērtējums ir negatīvs, tātad melnajām figūrām ir priekšrocība, savukārt, ja tas ir pozitīvs, tad attiecīgi baltās figūras ir vadībā.



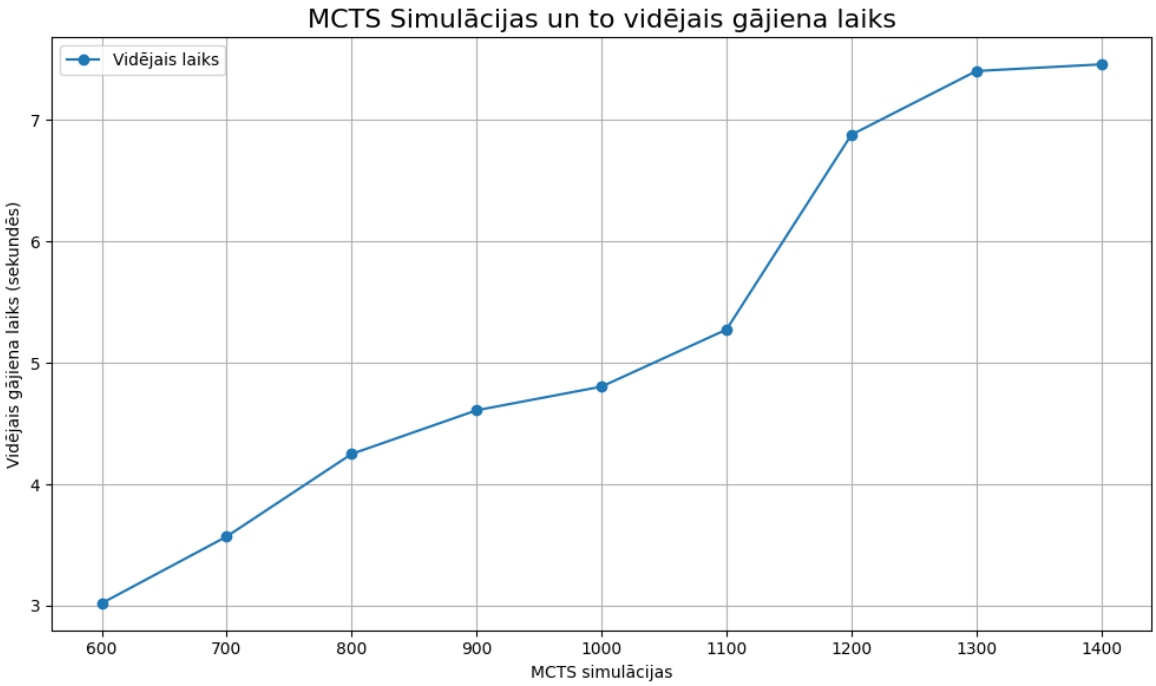
```
--------------------
Raw NN Suggestion -> Move: f1c4   | Confidence (Prob): 0.0709
  Stockfish eval for f1c4  : -8.57cp
----------
Starting MCTS search (600 sims, c_puct=1.4)...

--- MCTS Root Node Stats ---
Move   | Visits | Avg Value (Q)
-------|--------|---------------
h2h4   | 45     | -0.0671
b2b4   | 42     | -0.0770
g2g4   | 35     | -0.0802
a4a5   | 30     | -0.0838
f1c4   | 28     | -0.1278
g2g3   | 28     | -0.0778
b2b3   | 28     | -0.0828
f1a6   | 25     | -0.1026
f1e2   | 25     | -0.1118
b1a3   | 25     | -0.1109
h2h3   | 25     | -0.0789
f1d3   | 24     | -0.1294
a1a3   | 24     | -0.1139
f1b5   | 23     | -0.1119
c2c4   | 21     | -0.0768
MCTS Selection    -> Move: h2h4   | Visits: 45 (7.51%) | Q-Value: -0.0671
  Stockfish eval for h2h4  : -8.62cp
Total selection time: 4.56s
--------------------
```

3.1 att. MCTS algoritma darbības piemērs spēles laikā

Eksperimenta rezultātā tika izspēlētas vairākas spēles, starp kurām tika mainīts simulāciju skaits un figūru krāsa. Spēļu rezultāti bija šādi – neatkarīgi no krāsas, aģents, kurš spēlēja ar MCTS algoritmu un izmantoja simulācijas, kas bija robežās no 900 līdz 1100 vienmēr guva uzvaru pār aģentu, kurš neizmantoja MCTS algoritmu. Savukārt, situācijās, kad simulācijas bija robežās no 600 līdz 800 vai 1200 līdz 1400, aģents ātri vien guva materiālo pārsvaru, taču spēles beigās nespēja veikt šahmatu un ieciklējās uz šaha pieteikšanu pretinieka karaļa figūrai, taču pārējās tam pieejamās figūras atstāja novārtā, kas rezultējās ar bezjēdzīgiem un atkārtotiem gājieniem, kas nenoved pie uzvaras. Ņemot vērā faktu, ka šie eksperimenti ir veikti ar modeli, kas tika apmācīts uz 1,000,000 eksistējošām spēlēm, tad šādu rezultātu varētu skaidrot ar neironu tīkla nepietiekamu apmācību uz spēlēm, kas ir izvērtušās līdzīgi – liels materiālais pārsvars spēles beigās. Rezultātā modelis nespēj atrast gājienus par kuriem tas būtu ļoti pārliecināts, tāpēc izvēlas to, kas tam šķiet kā vislabākais, kas ir šaha pieteikšana pretinieka karalim, kaut arī beigās tas nenoved pie uzvaras.

MCTS algoritma ieviešana ne tika radīja pozitīvu efektu aģenta veiktspējā spēles laikā, taču attiecīgi palielināja arī gājiena veikšanas laiku atkarībā no simulāciju skaita. Kā iepriekš tika noskaidrots, optimālais simulāciju skaits ir rozbežās no 900 līdz 1100 simulācijām. Ja pirms tam aģents kurš darbojās izmantojot tikai neironu tīklu spēja veikt savu gājienu acumirklī, jeb tikai dažās milisekundēs, tad MCTS algoritma ieviešana būtiski palielināja šo gājiena veikšanas laiku, kas ir attēlots 3.1. attēlā. Izmantojot 800 simulācijas vidējais laiks, kas būs nepieciešams gājienam ir apmēram 4.24 sekundes, savukārt 1100 simulācijas prasīs 5.25 sekundes.



3.2. att. MCTS agloritma simulācijas un to vidējā gājiena laika grafiks

Sākotnējo eksperimentu rezultātā, liekot tiem pašiem modeļiem spēlēt vienam pret otru vairākas spēles un nemainot MCTS simulāciju skaitu vai figūru krāsu katram modelim, tika novērota tāda īpatnība, ka visas spēles lielākoties bija vienādas – identiski gājieni, to skaits un spēles rezultāts. No tā radās secinājums, ka neironu tīkls vienmēr dos vienu un to pašu rezultātu, kas ir politika un vērtība, ja tam liks apskatīt identiskus spēles gājienus, piemēram spēles sākumu, kurā jāviec pirmais gājiens melnajām, vai baltajām figūrām. Tieši tas pats notiek ar MCTS algoritmu – pie nemainīga simulāciju skaita, balastoties un neironu tīkla rezultātu, pēc savas formulas, tas vienmēr izvēlēsies tos pašus gājienus, kurus apskatīt un rezultātā izvēlēties vienu kā

labāko. Lai situāciju labotu, MCTS algoritmā tika ieviestas nelielas izmaiņas – situācijā, kad aģents veic savu pirmo gājienu spēlē, tas izvēlas nevis nevis to gājienu, ko MCTS uzskata par labāko, taču izvēlas nejaušā kārtā kādu no trīs labākajiem, ko algoritms piedāvā. Rezultātā, sākotnējā problēma atrisinājās un spēles vairs nebija identiskas. Tomēr līdz ar to parādījās jauna problēma – ja sākumā, kad aģents vienmēr izvēlējās labākos gājienus, tad tas nesagādāja tam grūtības gūt uzvaru spēlē, tad momentā, kad tika ieviests nejaušības princips sākotnējo gājienu izvēlē, aģentam bija nepieciešams veikt daudz vairāk gājienu lai gūtu uzvaru, vai nonāktu līdz neizšķirtam rezultātam. Tas ir skaidrojams ar to, ka iemesls, kāpēc neironu tīkls iesaka konkrētu gājienu kā labāko ir tāpēc, ka savas apmācības procesā tas ir izskatījis daudz vairāk veiksmīgu spēļu tieši ar šāda gājiena veikšanu, nekā veiksmīgas spēles ar citiem gājieniem. Rezultātā, ja nejaušas izvēles procesā tiek izvēlēts, piemēram, trešais labākais gājiens, tas nozīmē, ka neironu tīklam jau būs daudz grūtāk veikt turpmākos gājienus pietiekami pārliecinoši un stratēģiski. Tas noved arī pie secinājuma par to, cik būtiski ir spēles sākotnējie gājieni un kā tie var ietekmēt turpmāko spēles gaitu un tās rezultātu.

3.4. Eksistējošo spēļu aģents

Lai apmācītu aģentu no eksistējošām spēlem tiek izmantota lichess 2024. gada decembra atvērtā datubāze, kas satur 96587411 novērtētas, dažāda reitinga spēles .pgn formātā. Jāņem vērā fakts, ka reitings, kas ir piešķirts lichess spēlēm, izmanto Glicko-2 vērtēšanas sistēmu, kas būtiski atšķiras no Elo reitinga sistēmas, ko izmanto FIDE. Daļai no spēlēm ir pieejams arī Stockfish gājienu novērtējums spēles analīzes nolūkā izteikts bandinieku simtdaļās (angļu val – *centipawns*), un kā vienu spēles piemēru, kas atrodas šajā datubāzē ir iespējams aplūkot 3.3. attēlā.

```
[Event "Rated Blitz game"]
[Site "https://lichess.org/grNHa3ul"]
[Date "2024.12.01"]
[Round "-"]
[White "Nooobasikkk"]
[Black "crimson-chin"]
[Result "0-1"]
[UTCDate "2024.12.01"]
[UTCTime "00:00:10"]
[WhiteElo "1507"]
[BlackElo "1388"]
[WhiteRatingDiff "-7"]
[BlackRatingDiff "+8"]
[ECO "C47"]
[Opening "Four Knights Game: Halloween Gambit"]
[TimeControl "180+0"]
[Termination "Normal"]

1.e4 (-0.18) 1...e5 (+0.21) 2.Nf3 (-0.13) 2...Nc6 (+0.17) 3.Nc3 (-0.11) 3...Nf6 (+0.08)
4.Nxe5 (+2.41) 4...Nxe5 (-1.57) 5.d4 (+1.91) 5...Nc6 (-0.54) 6.d5 (+0.53) 6...Ne5
(-0.50) 7.f4 (+0.50) 7...Ng4 (+0.00) 8.h3 (-0.12) 8...Nxe3 (+2.92) 9.Bxe3 (-2.96)
9...Bb4 (+2.95) 10.e5 (-2.52) 10...Bxc3+ (+3.52) 11.bxc3 (-3.31) 11...Ng8 (+3.94)
12.Bc4 (-3.57) 12...Qh4+ (+3.75) 13.Kf1 (-3.02) 13...Ne7 (+3.31) 14.d6 (-2.21)
14...cxd6 (+2.08) 15.exd6 (-1.98) 15...Nf5 (+2.77) 16.Qg4 (+6.47) 16...Nxe3+ (-6.32)
17.Ke2 (+8.05) 17...Nxg4 (-8.07) 18.hxg4 (+7.98) 18...Qxg4+ (-8.15) 19.Kf2 (+8.80)
19...Qf4+ (-8.98) 20.Kg1 (+9.05) 20...Qxc4 (-8.90) 21.Rf1 (+9.62) 21...O-O (-8.01)
22.Rf3 (+9.37) 22...Qa2 (-8.08) 23.Rh3 (+8.57) 23...h6 (-8.33) 24.Rg3 (+9.13) 24...Qc2
(-8.97) 25.Rxh6 (+12.10) 25...Qc1+ (-11.70) 26.Kf2 (+11.41) 26...Qxh6 (-10.68) Result:
0-1
```

3.3. att. Lichess datubāzes pabeigtas spēles piemērs

Tām spēlēm, kurām Stockfish novērtējums datubāzē nav pieejams, tiks izmantota lokāli lejupielādēta Stockfish 17 AVX2 versija ar noteiktu novērtēšanas dziļumu 6, kas dos šo gājienu novērtējumu.

3.4.1. Spēļu lasīšana

Lai iespējotu aģenta apmācību ar apjomīgu datu kopu, ir nepieciešams izmantotās spēles priekšlaicīgi iekodēt attiecīgajā formātā un saglabāt diska krātuvē, jo pilnā datu kopa pārsniedz pieejamo operatīvās atmiņas (RAM un VRAM) apjomu.

Spēļu kodēšana notiek pēc 3.2. nodaļā apstrakstītā principa, izmantojot Python izveidotas funkcijas.

Sākotnēji eksistējošā aģenta apmācība tika veikta, spēles nolasot no PGN faila, tās priekšlaicīgi iekodējot un tad visas glabājot operatīvajā atmiņā. Šāda pieejas izrādījās neefektīva ar pieejamajiem skaitļošanas resursiem, jo operatīvajā atmiņā bija iespējams glabāt līdz 300000 spēlēm un tās netika saglabātas, beidzot modeļa

apmācību, kā rezultātā katru reizi, apmācot jaunu modeļa versiju, bija nepieciešams no jauna veikt spēļu kodēšanu. Ierobežoto skaitļošanas resursu un neefektīvu šo resursu izmantošanas dēļ tika veiktas izmaiņas, rezultātā nonākot pie gala risinājuma – datu glabāšanai diska krātuvē un datu nolasīšanai apmācības laikā.

Spēļu saglabāšanai diska krātuvē tiek izmantots HDF5 (Hierarchical Data Format) hierarhiskais datu formāts, kas ir optimizēts ātrai lielapjoma datu piekļuvei. Formāta integrācijai ar Python programmēšanas valodu tiek pielietotas h5py un hdf5plugin bibliotēkas. hdf5plugin bibliotēka sniedz iespēju izmantot papildu saspiešanas algoritmus, tostarp LZ4, kas tika izvēlēts salīdzinot to ar h5py bibliotēkas noklusējuma pieejamo LZO algoritmu. LZ4 nodrošināja aptuveni 5 reizes ātrāku atspiešanas veiktspēju, būtiski paātrinot apmācības procesu.

Datu saspiešanas algoritms ir būtiska spēļu glabāšanas daļa, jo būtiski samazina datu pārraides apjomu starp disku un operatīvo atmiņu – I/O operācijas.

HDF5 failā dati tiek hierarhiski strukturēti un iedalīti vairākās datu kopās:

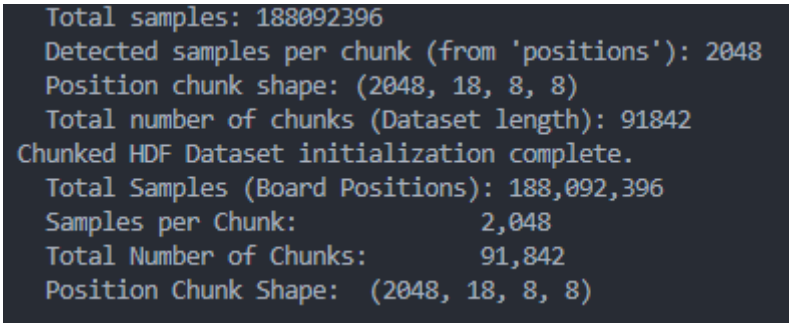
1) Pozīcijas – satur iekodētās 18 plaknes kā NumPy masīvs ar formu (18,8,8), rezultātā veidojot (N,18,8,8) kopu, kur N ir kopējais pozīciju skaits failā
2) Gājieni – satur pozīcijai iekodēto nākamo gājienu (N, 1) formā.
3) Vērtības – satur pozīcijas novērtējumu no Stockfish dziņa, normalizēts no -1 līdz 1, to glabājot (N,1) formā

Datu kopas tiek glabātas iedalītos blokos (angļu val. *chunks*) ar definētu izmēru 2048. Bloku glabāšana iespējo ātrāku datu nolasīšanu no faila un datu saspiešana tiek atsevišķi piemērota katram datu blokam.

Spēļu kodēšana notiek paralēli ar norādīto plūsmu skaitu, galvenajam procesam iteratīvi lasot spēles no norādītā PGN faila, apstrādājot norādīto spēļu diapazonu. Spēļu nolasīšanai tiek izmantota “chess” bibliotēkas pieejamā funkcija “read_game”, lai atvieglotu eksistējošo spēļu gājienu lasīšanu.

Nolasītās spēles tiek iedalītas pārējām norādītajām plūsmām, tām veicot pozīciju un gājienu kodēšanu, kā arī novērtējuma noteikšanu vai nolasīšanu. Apstrādātie dati tiek atgriezti atpakaļ galvenajam procesam, kas secīgi datus pievieno attiecīgajām datu kopām iepriekš izveidotajā HDF5 failā.

Labākā aģenta versijā tika izmantots 2,5 miljonu iepriekš iekodētu spēļu fails, kura izmēri ir redzami 3.4. attēlā:



3.4. att. Gala apmācībā izmantotā HDF5 faila izmēri

Šis gala izveidotais fails sastāv no 188 miljoniem iekodētiem gājieniem, kopā veidojot gandrīz 92 tūkstošus bloku, lasīšanas un kodēšanas procesam aizņemot apmēram 23 stundas, gala rezultātā iegūstot 6,19GB lielu failu.

3.4.2. Aģenta apmācība

HDF5 fails tiek integrēts ar PyTorch Dataset klasi, sniedzot iespēju nolasīt datus no failā esošajiem blokiem. Viens HDF5 failā esošais bloks tiek uzskatīts kā viena vienība. Bloku skaits tiek iegūts, dalot kopējo failā esošo gājienu skaitu ar katrā blokā esošo gājienu skaitu, vienā blokā esot 2048 gājieniem. HDF5 fails tiek atvērts, izmantojot hdf5_worker_init funkciju, kamēr chunked_collate apvieno dotos blokus vienā tenzoru krājumā un nosūtot to apmācībai uz videokarti.

Lai paātrinātu apmācību, tiek veiktas vairākas vienkāršākas optimizācijas, kā NVIDIA Tensor kodolu izmantošana, zemākas precizitātes float datatipu izmantošana.

Veicot modeļa apmācību ar 1000000 spēlēm, viens epohs ar 12 miljoniem parametru aizņem apmēram 16 minūtes, ar pilnīgu videokartes noslodzi, 2 procesora plūsmu noslodzi un apmēram 5GB izmantotās operatīvās atmiņas.

Labākais aģents, kas tika iegūts ir apmācīts ar 2 miljoniem spēļu, starp kurām zemākais Glicko-2 reitings ir 2000. Šī modeļa apmācība aizņēma 9 epohus un aptuveni 12,8 stundas.

3.5. Pašspēles aģents

Pašspēles aģenta apmācība krasi atšķiras no eksistējošo spēļu aģenta, jo viss apmācības process balstās uz aģenta spēju apmācīt sevi, nepaļaujoties uz eksistējošiem datiem. Pašspēles aģentam nav pieeja jau veiktam gājienu novērtējumam, kā, piemēram, Stockfish pozīciju novērtējums jau izspēlētām partijām, kas padara tā apmācību grūtāku, jo apmācības laikā visiem veiktajiem gājieniem ir viens novērtējums, kas atkarīgs no spēles rezultāta – 1 par baltās krāsas uzvaru, 0 par neizšķirtu un -1 par melnās krāsas uzvaru.

Spēļu lasīšana pašspēles aģentam sakrīt ar eksistējošo spēļu aģenta spēļu lasīšanu, kas ir norādīta 3.4.1 nodaļā.

3.5.1. Spēļu ģenerēšana

Ģenerējot spēles, tika pieņemts lēmums neizmantot MCTS un paļauties tikai uz neironu tīklu, citādi spēļu ģenerēšana ir daudzreiz lēnāka. Salīdzinot gājiena veikšanas ātrumu, izmantojot spēļu ģenerēšanu ar 100 MCTS simulācijām gājiena veikšana aizņem apmēram 10 reizes vairāk laika, kas noved pie nepietiekama spēļu daudzuma, lai aģents spētu uzlabot savas prasmes. Papildus tika ierobežots maksimālais gājienu skaits uz 100, rezultātā iegūstot 20-25 sekunžu ģenerēšanas laiku 1 spēlei uz 1 procesora plūsmas, parasti vienlaicīgi izmantojot 15 procesora plūsmas.

Stundas laikā ir iespējams ģenerēt apmēram 3 tūkstošus spēļu, iegūstot apmēram 150 tūkstošus dažādu pozīciju, no kurām aģents var mācīties.

Spēļu ģenerēšanas uzsākšana un apturēšana notiek manuāli, to apturot, kad ir sasniegts vēlamais spēļu skaits, tām tiekot saglabātām HDF5 failā, formātam sakrītot ar 3.4.1. nodaļā aprakstīto spēļu lasīšanu, kur spēles tiek saglabātas blokos ar definētu izmēru 2048.

3.5.2. Aģenta apmācība

Aģenta apmācība notiek pēc vēlamā spēļu skaita sasniegšanas ģenerēšanas laikā. Spēles, tāpat kā eksistējošo spēļu aģentam, tiek pakāpeniski nolasītas no izveidotā HDF5 faila un aģents tiek apmācīts līdz tiek sasniegts maksimālais norādīto epohu skaits, vai sasniegts pacietības mērs. Eksistējošo spēļu aģents tiek apmācīts ar

ievērojami augstāku pacietības mēru un maksimālo epohu skaitu, jo spēļu skaits ir daudzreiz mazāks, nekā eksistējošo spēļu aģentam, maksimālo epohu skaitam parasti tiekot iestatītam uz 100 maksimālajiem epohiem ar pacietības mēru 20 – ja 20 epohu laikā netiek iegūts labāks modelis, tad tiek ņemts labākais atrastais modelis.

Apmācības sākumā tika novērots ļoti augsts neizšķirtu partiju skaits, kā arī ļoti liels atkārtotu gājienu skaits, kas tika risināts, pielietojot temperatūru gājienam – iespēju paaugstināšana kādam no neoptimālajiem gājieniem. Gājienu temperatūra tika pielietota pirmajos 20 vai 30 gājienos, atkarībā no pašspēles aģenta iterācijas. Šāds risinājums palīdzēja nedaudz samazināt neizšķirtu partiju skaitu, kaut gan apmēram 80% no partijām tāpat beidzās ar neizšķirtu, neatkarīgi no apmācības laika.

Neizšķirto partiju skaits tika krasi samazināts, novirzoties no AlphaZero stila pilnīgas pašspēles, ieviešot divas nelielas heiristikas – tika palielināta aģenta iespēja pieteikt šahmatu, ja tas ir iespējams, un tika samazināta iespēja atkārtot to pašu gājienu 2 vai 3 reizes, tādējādi lielākoties novēršot neizšķirtu partiju ar 3 gājienu atkārtošanos.

Šī novirze no pilnīgas pašspēles aģenta samazināja neizšķirtu partiju skaitu uz aptuveni 55%, no kuriem gandrīz visas partijas ieguva šādu rezultātu, jo tika sasniegts maksimālais gājienu skaits. Maksimālā gājienu skaita palielināšana gan tālāk nesniedza statistiski novērojamu rezultātu, jo aģents, lai arī tas neatkārtoja tos pašus 3 gājienus, sāka atkārtot garākas gājienu ķēdes, netiekot novērotam tālākam spēles progresam pat ar 300 maksimālajiem gājieniem.

Kopēji visu pašspēles aģenta iterāciju apmācība aizņēma apmēram 5 stundas, papildus aptuveni 38 stundām tiekot patērētām, ģenerējot apmācības spēles.

Kopumā tika ģenerētas 150000 spēles, labāko sniegumu uzrādot priekšpēdējai aģenta versijai, kas kopumā tika apmācīta ar 30000 mazāk spēlēm, nekā pēdējā versija, kas uzrādīja nedaudz vājākus rezultātus.

3.6. Salīdzināšanas metodoloģija

Lai izvērtētu šaha spēles aģentu spējas, tie tiek salīdzināti, spēlējot vienam pret otru 24 spēļu garumā. Abiem aģentiem tiks izmantots mainīgs MCTS simulāciju skaits ar soli 100, robežās no 600 līdz 1100 simulācijām, kur katrā solī tiks izspēlēti 2 spēļu pāri.

Abiem aģentiem tiek iedalīti vienādi skaitļošanas resursi – pilnīga piekļuve visām 16 procesora plūsmām, pilnīga piekļuve visai videokartei un atvēlēti 28GB operatīvās atmiņas, kas piešķirta WSL. Starp aģentiem vispārējais uzvarētājs ir tas aģents, kurš ir ieguvis lielāku punktu skaitu. Punktu sadale seko šaha turnīra noteikumiem, kur uzvarētājs iegūst 1 punktu un zaudētājs iegūst 0 punktu. Neizšķirtas spēles gadījumā abi aģenti saņem 0,5 punktus.

Izspēlētās partijas tiek saglabātas .pgn failā, kurā ir attēloti dati par izspēlētās partijas rezultātu, redzami visi veiktie gājieni, un izmantotais MCTS simulāciju skaits, kā arī Stockfish dziņa pozīcijas novērtējums.

Aģenti pēc katras spēles tiek vērtēti un tiem piešķirti un koriģēti elo reitingi, izmantojot 1.7. nodaļā aprakstītās formulas (1.2) un (1.3). Sakarā ar to, ka sākotnēji tiem nebūs objektīva reitinga, tad pirms spēļu sākšanas katram tiks piešķirts reitings "800", kas atbilst iesācēja līmenim.

Labākais no izstrādātajiem aģentiem tiek salīdzināts ar Stockfish 17 AVX2 dzini ar dažādu Elo reitingu, izmantojot mainīgu MCTS simulāciju skaitu ar soli 200, robežās no 800 līdz 1600 simulācijām, katrā solī tiekot izspēlētiem vismaz 2 spēļu pāriem. Stockfish dzinim gājiena veikšanai tiks dota 1 procesora plūsma un tā gājiena laiks tiks noapaļots un pielīdzināts MCTS aģenta 1600 simulāciju veikšanas laikam.

Gājienu kvalitāte tiks salīdzināta ar Stockfish 17 AVX2 dziņa versiju.

Aģentu savstarpējās spēles tiek veiktas, izmantojot python-chess bibliotēkas atbalstu, vizualizējot aģentu veiktos gājienus, vienlaikus sniedzot iespēju ierobežot aģentu pieejamos skaitļošanas resursus un maksimālo gājiena laiku. Izstrādātie aģenti tiks integrēti ar python-chess bibliotēkā izmantoto universālo šaha interfeisa (UCI) protokolu, izmantojot izstrādātu risinājumu, kas pārvērš iegūto politiku derīgos UCI gājienos.

Lai salīdzinātu pašas apmācības metodes, kas tika pielietotas uz šiem aģentiem – pašspēle un vēsturisko spēļu analīze, un noskaidrotu to stiprās un vājās puses, tiek novērtēts un salīdzināts ne tikai rezultāts, ko tas ir devis (aģentu veiktspēja), bet arī pats process un resursi, ko tas prasīja. Primāri tiek aplūkots un salīdzināts laiks, kas tika patērēts aģentu apmācības procesā, spēļu ģenerēšanas procesā pašspēles aģentam un spēļu lasīšanas un kodēšans procesā vēsturisko spēļu analīzes aģentam.

3.7. Šaha aģentu salīdzinājuma rezultāti

Pēc tam kad abiem aģentiem ir iegūtas pēdējās, jeb labākās versijas, tie tiek salīdzināti savā starpā liekot spēlēt vienam pret otru, lai nosakidrotu, kurš no tiem ir sasniedzis lielāku meistarības līmeni un cik ļoti to spēlēšanas veidi atšķiras. Iegūtie rezultāti ir sekojoši - netkarīgi no MCTS simulāciju skaita, eksistējošo spēļu aģents vienmēr guva uzvaru, kā rezultātā no 24 spēlēm tas ir ieguvis 24 punktus.

Kad aģenti uzsāk spēli, šo visu spēles procesu ir iespējams novērot terminālī, kurā ir redzams ne tikai šaha spēles laukums un aģentu veiktie gājieni, bet arī vairāki vērtīgi analīzes dati, kā piemēram, aģenta pārliecība par veiktajiem gājieniem, MCTS izskatītie gājieni un Stockfish evaluācija dotajam spēles stāvoklim, ko uzskatāmi ilustrē 3.4. attēls.

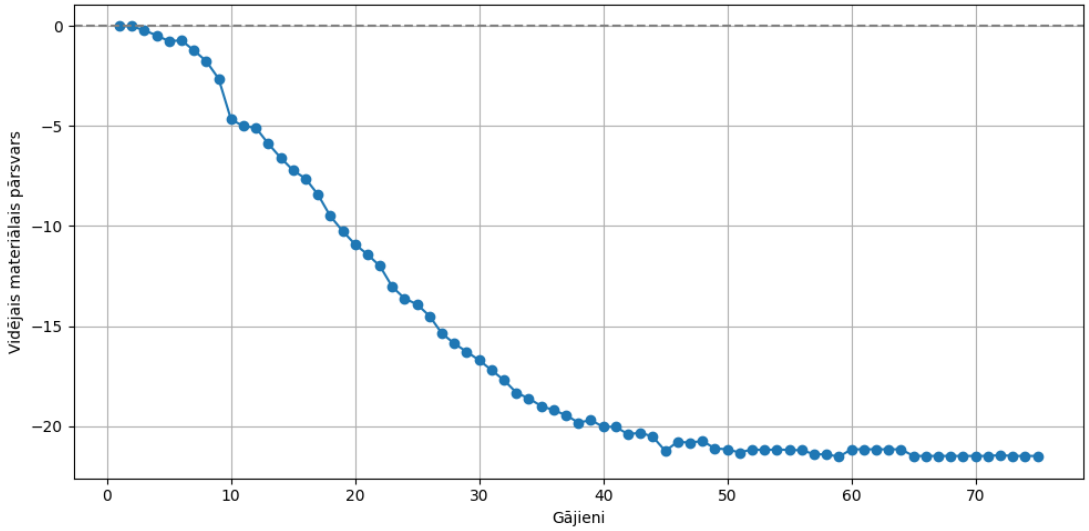
```
Move 19
r . b . k b . r
p p . p . p p p
. . . . p n . .
. B . . . . . .
P . . . . . . q
. . . R . . . .
. P n P . P P K
. . . . . . . .
Engine 1 (W - selfPlay-v7-ep02-acc01645.pth) thinking (MCTS)...
--------------------
Raw NN Suggestion -> Move: h2g1   | Confidence (Prob): 0.3739
  Stockfish eval for h2g1  : Mate -6
----------
Starting MCTS search (600 sims, c_puct=1.4)...

--- MCTS Root Node Stats ---
Move   | Visits | Avg Value (Q)
-------|--------|--------------
h2g1   | 511    | -0.2852
d3h3   | 88     | -0.3037
MCTS Selection    -> Move: h2g1   | Visits: 511 (85.31%) | Q-Value: -0.2852
  Stockfish eval for h2g1  : Mate -6
Total selection time: 4.84s
```

3.4. att. Vizuāls spēles piemērs terminālī

Pēc tam kā tika pavērota vairāku spēļu gaita, tika fiksētas vairākas sakarības, kur, piemēram, eksistējošo spēļu aģentam joprojām ir tendence neizmantot izdevības situācijās, kur ir iespējams pietekt šahu pretinieka karalim un gūt uzvaru. Šis aģents ir

iemācījies un bieži vien pieturas pie principa, kur tas sākotnēji liek prioritāti uz materiālā pārsvara gūšanu, atstājot pretinieku ar karali un pavisam dažiem bandiniekiem uz spēles laukuma, un tikai tad fokusēties uz uzvaras gūšanu. Šī sakarība ir attēlota 3.5 attēlā, kur, skatoties no pašspēles aģenta perspektīvas, līknes veidā ir parādītas vidējās materiālā balansa izmaiņas visu spēļu laikā. Jau sākot no paša spēles sākuma pašspēles aģents strauji zaudē materiālo balansu un tikai retos gadījumos spēj situāciju mazliet labot. Dotajā attēlā materiālais balanss ir attēlots balstoties uz figūru vērtībām un nav pārveidots bandinieku simtdaļās.



3.4 att. Materiālais balanss spēļu laikā no pašspēles aģenta perspektīvas

Abiem aģentiem tika aprēķināts Elo reitings, kas pēc 24 veiktajām spēlēm izskatījās šādi:

- Pašspēles aģenta Elo retings – 598
- Eksistējošo spēļu aģenta Elo reitings – 1002

Ņemot vērā to, ka pašspēles aģents 24 spēļu garumā nespēja gūt nevienu uzvaru, kā arī izcīnīt neizšķirtu rezultātu, tad attiecīgi tā Elo reitings no sākotnējā vērtējuma 800 nokritās par 202 punktiem, savukārt pretiniekam attiecīgi vērtējums paaugstinājās par 202 punktiem.

Noskaidrojot to, ka abi aģenti acīmredzami atrodas pavisam dažādos meistarības līmeņos, tad, lai precīzāk spētu novērtēt eksistējošo spēļu aģenta spējas, tiek veiktas 36 spēles pret Stockfish dzini. Izspēlēto partiju rezultāti ir atspoguļoti 3.1. tabulā, kur redzams uzvaru, zaudējuma un neizšķirtu spēļu attiecība skatoties no

eksistējošo spēļu aģenta perspektīvas. Stockfish Elo reitings tika ierobežots, izmantojot UCI_LimitStrength komandu kopā ar UCI_Elo.

3.1. tabula

Spēļu rezultāti no eksistējošo spēļu aģenta perspektīvas

MCTS simulāciju skaits	Stockfish Elo	Uzvara	Neizšķirts	Zaudējums
800	2000	4	0	0
1000	2000	4	0	0
800	2250	4	0	0
1000	2250	3	1	0
800	2500	3	1	0
1000	2500	4	0	0

Analizējot iegūtos rezultātus var secināt, ka eksistējošo spēļu aģentam nesagādāja grūtības gūt uzvaru spēlēs pret Stockfish dzini ar 2000 Elo reitingu, kā arī gandrīz tikpat pārliecinošas bija spēles pret 2250 un 2500 Elo dzini, ar diviem izņēmumiem, kur tika panākti divi neizšķirti rezultāti.

Lielākas grūtības jau sāka sagādāt Stockfish ar 2750 Elo reitingu, kur kaut arī tika gūtas uzvaras, bet jau biežāk sāka parādīties neizšķirti rezultāti un pat zaudējumi, kas ir attēlots 3.2. tabulā. Kopumā pret Stockfish 2750 Elo dzini tika izspēlētas 100 spēles, kur, izmantojot MCTS simulāciju skaitu ropbežās no 800-1600 ar soli 200, katrā simulāciju skaitā tika izspēlēti 5 spēļu pāri. Stockfish gājiena veikšanai tika dotas 10 sekundes, kas ir pielīdzināms vidējam gājiena laikam ar eksistējošo spēļu aģentu, izmantojot 1600 MCTS simulācijas, paralēli spēlējot 8 partijas.

3.2. tabula

Spēļu rezultāti no eksistējošo spēļu aģenta perspektīvas pret Stockfish 2750 Elo

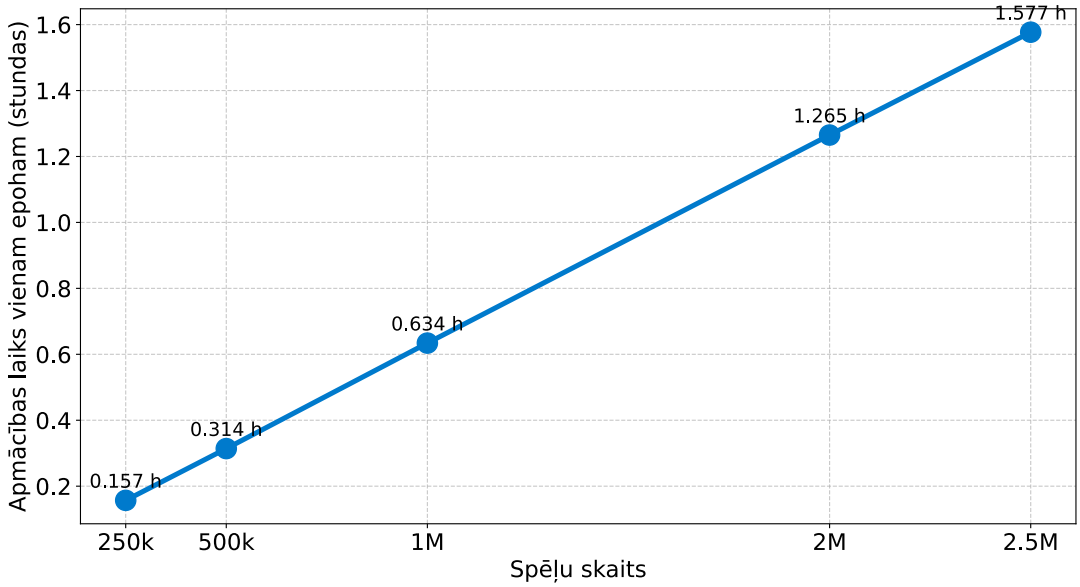
MCTS simulāciju skaits	Uzvara	Neizšķirts	Zaudējums
800	10	3	7
1000	8	5	7
1200	5	6	9
1400	11	6	3
1600	5	9	6

Analizējot iegūtos rezultātus var secināt, ka uzvaru, neizšķirtu un zaudējuma attiecība ir ļoti līdzīga savā starpā, kas nozīmē, ka abi aģenti atrodas vienā meistarības līmenī un ir pienācīgi pretinieki viens otram. Saistība starp uzvaru skaitlu un MCTS simulāciju paaugstināšanos nav novērota.

Pēc izspēlētajām partijām eksistējošo spēļu aģentam tiek atkal aprēķināts Elo reitings. Balstoties uz iepriekšējo spēļu rezultātiem, aprēķinos tiek pieņemts, ka aģenta sākotnējais Elo reitings ir 1002, kas pēc visiem šiem mačiem ar Stockfish ir vairāk kā dubultojies un pēc aprēķiniem šobrīd ir 2679.

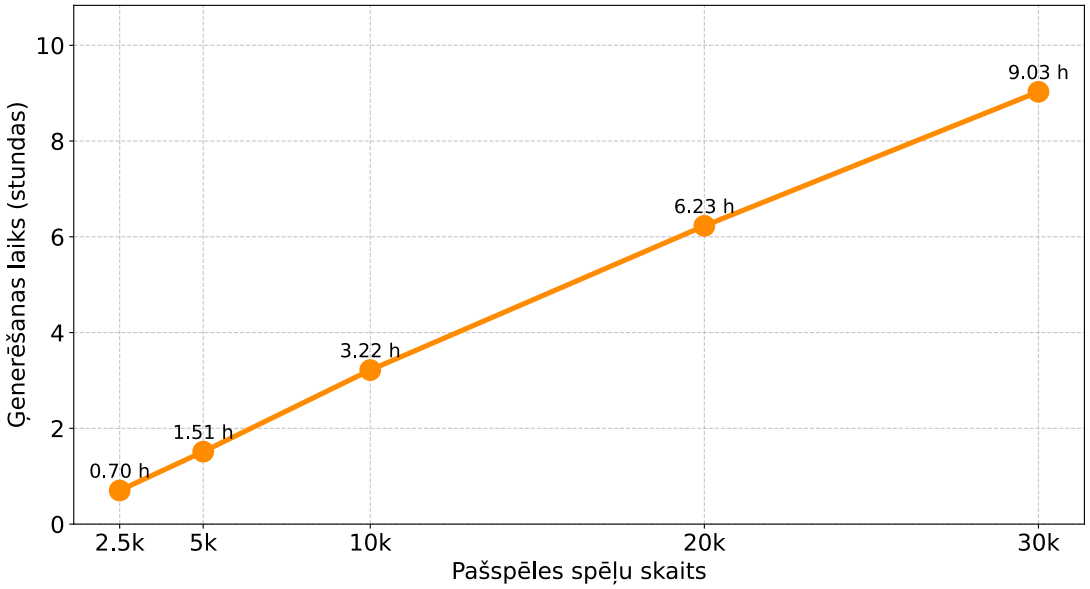
3.8. Apmācības metožu salīdzinājums

Skatoties uz eksistējošo spēļu apmācības laiku, atkarībā no spēļu skaita, kā redzams 3.5 attēlā, ir novērojama lineāra sakarība, kur apmācības laiks dubultojas, dubultojoties spēļu skaitam.



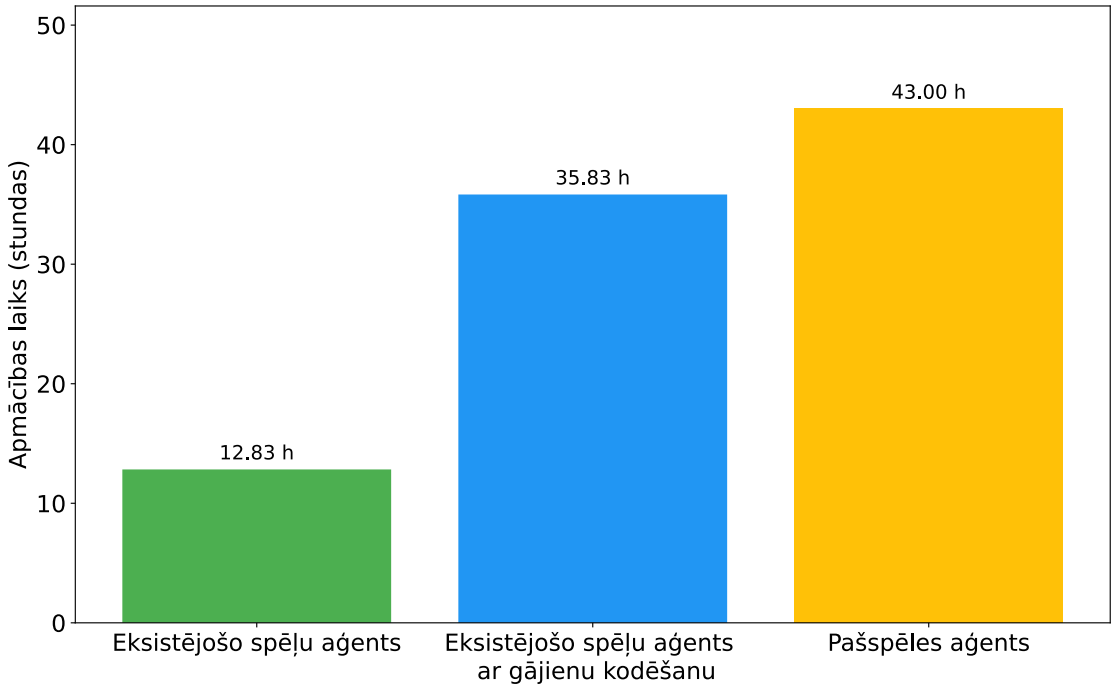
3.5 att. Apmācības laika atkarība no spēļu skaita eksistējošo spēļu aģentam

Skatoties uz pašspēles aģentu spēļu ģenerēšanas laiku, ir novērojams, ka laiks palielinās, palielinoties ģenerētajām spēlēm, lai arī svārstības, kā redzams 3.6 attēlā, ir lielākas. Pašspēles spēļu skaits ir neparedzamāka vērtība, jo gājienu skaits var un bieži vien atšķirās, atkarībā no izspēlētajā partijā veiktajiem gājieniem.



3.6 att. Spēļu ģenerēšanas laiks atkarībā no ģenerēto spēļu skaita pašspēles aģentam

Skatoties uz pēdējo modeļu versiju gala apmācības laiku, kā redzams 3.7 attēlā, ir skaidri novērojams, ka pašspēles aģents ir aizņēmis lielāku apmācības laiku, vienlaikus sasniedzot daudz vājāku rezultātu. Neieskaitot gājienu kodēšanu (kas ir vienreizēja darbība, jo iekodētos gājienus pēc tam ir iespējams vairākkārt izmantot), eksistējošo spēļu aģenta kopējais apmācības laiks bija 3,35 reizes zemāks, bet, ieskaitot kodēšanu – 1,2 reizes zemāks.



3.7 att Labāko aģentu kopējo apmācības laiku salīdzinājums

3.9. Rekomendācijas apmācības metodes izvēlei

Atkarībā no tā, kāds ir izstrādātāja mērķis, atbilstošas apmācības metodes izvēle šaha dziņai izstrādei ir fundamentāls pamats turpmākajam izstrādes procesam.

Situācijā, kad darba mērķis ir eksperimentēt un redzēt straujus uzlabojumus šaha spēles aģenta veiktspējā, tad vislabākā apmācības metode, ko pielietot būtu vēsturisko spēļu analīze. Publiski pieejamās datubāzes ar pabeigtajām šaha spēlem un to dažādība dod lielu priekšroku un izvēles brīvību, lai noteiktu kādā meistarības līmenī ir vēlme savu aģentu apmācīt. Ar nepieciešajām optimizācijām, kas ļauj efektīvi izmantot pieejamos skaitļošanas resursus un optimālas un ātras programmēšanas valodas izvēli, piemēram C++, šī aģenta izstrāde līdz vēlamajam meistarības līmenim būtu iespējama vēl īsākā laikā, jo tiktu daudzkārt paātrināts laikietilpīgākais process – gājienu priekšlaicīga kodēšana. Darba laikā eksperimentālos nolūkos tika apskatīta iespēja gājienus iekodēt, izmantojot Rust programmēšanas valodu, kas pirmajos testos sniedza apmēram 4 reizes lielāku spēļu nolasīšanas ātrumu, salīdzinot ar gala rezultātā izmantoto Python versiju, pierādot to, kad ar lielāku laika ieguldījumu ir iespējams

iegūt vēl vairāk spēļu, kuras varētu pielietot apmācības procesā, potenciāli sasniedzot augstākus rezultātus.

Savukārt, situācijā, kad izstrādātāja mērķis ir pētīt aģenta attīstību situācijā, kur tas mācās apgūt šaha spēli, izmantojot tikai šaha spēles noteikumus,tad piemērotākā apmācības metode ir izmantot pašspēles. Tomēr jārēķinās ar to, ka spēļu ģenerēšana, īpaši situācijās, ja papildus tiek izmantots kāds pārmeklēšanas algoritms, piemēram MCTS, tad tas aizņems ievērojami vairāk laika.

REZULTĀTI UN SECINĀJUMI

Bakalaura darba rezultātā tika izveidoti divi šaha spēles aģenti python programmēšanas valodā, no kuriem katrs tika apmācīts, izmantojot savādāku apmācības metodi – pašspēles un vēsturisko spēļu analīzi. Balstoties un analītiskās daļas rezultātiem, aģentu izstrādei tika izvēlēts izmantot MCTS pārmeklēšanas algoritmu, sakarā ar tā piemērotību ierobežotiem skaitļošanas resursiem un konvolucionālo neironu tīklu, kas ir pierādījis savu veiktspēju jau citos šaha aģentos, kā piemēram, Lc0.

Ir implementētas vairākas opcijas tam, kāda veida partijas iespējams izspēlēt starp aģentiem – MCTS aģents pret neironu tīklu, pašspēles aģents pret eksistējošo spēļu aģentu, spēle pret Stockfish aģentu, kā arī iespējams spēlēt pašam pret kādu no izvēlētajiem aģentiem terminālī.

Visas izspēlētās partijas tiek saglabātas .pgn failos un ir pieejamas apskatei GitHub (1. pielikums). Lai partiju analīze būtu ērtāka, ir iespējams .pgn failu importēt lichess.org mājaslapā, sadaļā Analysis board, kur visu spēles gaitu ir iespējams redzēt uz spēles laukuma un aplūkot dažādus spēles stāvokļus.

Ir izveidota salīdzināšanas metodoloģija šaha aģentiem un apmācības metodēm, kā arī iegūtie salīdzinšāsnas rezultāti atspoguļoti grafiku un tabulu veidā.

Bakalaura darba galvenie secinājumi ir:

1. MCTS algoritma ieviešana būtiski uzlaboja aģenta veiktspēju, kur spēlēs pret to pašu aģentu, kurš izmanto tikai neironu tīklu, aģents ar MCTS vienmēr guva uzvaru.
2. Neironu tīkla vērtības noteikšanas daļā lineāro slāņu pievienošana sniedza novērojami precīzāku pašreizējās pozīcijas novērtējumu, salīdzinot ar vienkāršu 64 1x1 filtru pielietošanu, kam sekoja ReLu aktivācijas funkcija. Tāpat reziduālo bloku palielināšana no 10 līdz 20 sniedza būtisku dziņa veiktspējas uzlabojumu, kaut arī apmācības laiks arī attiecīgi dubultojās.
3. Eksistējošo spēļu aģents efektīvi tiek galā ar spēlēm pret augsta un zema līmeņa pretiniekiem. Ar ilgāku apmācības laiku, tika novērots, ka aģents paliek agresīvāks – biežāk veic figūru apmaiņu. Savukārt ar zemu apmācības laiku, aģents spēlēja ļoti pasīvi, ļoti bieži neņemot pretinieka figūras, pat ja tās netiek citādi sargātas.

4. Eksistējošo spēļu aģents labāko rezultātu uzrāda pēc ap 10-20 spēles epohiem, ilgākam apmācības laikam atstājot negatīvu ietekmi uz rezultātu. Šis aģents uzrāda labākus rezultātus, tam tiekot apmācītam ar augstāku spēļu skaitu.
5. Eksistējošo spēļu aģents izspēlēja 24 spēles pret pašspēles aģentu un 124 spēles pret Srtockfish aģentu ar mainīgu Elo reitingu. Rezultāta tika aprēķināts eksistējošo spēļu aģenta Elo reitings, kas šobrīd ir 2679 robežās.
6. Pašspēles aģents nav spējis sasniegt pietiekami augstu meistarības līmeni, lai konkurētu pret eksistējošo spēļu aģentu. Tas ir skaidrojams ar nepietiekamo spēļu skaitu, ko tas ir uzģenerējis, lai mācītos no tām un ilgo laiku, ko šis process patērē, kur salīdzinot ar eksistējošo spēļu aģentu (neieskaitot spēļu kodēšanas laiku, jo tas ir vienreizējs process), pašspēles aģenta apmācība un spēļu ģenerēšana aizņēma 3,35 reizes vairāk laika.
7. Iemesls tam, kāpēc modeļu apmācības process aizņem ievērojami daudz laika ir saistīts ne tikai ar ierobežotajiem skaitļošanas resursiem, kas tiek izmantoti, bet arī ar neoptimālu programmēšanas valodas izvēli, kas šajā gadījumā ir Python, salīdzinājumā ar, piemēram, C++. Sākotnēji tika izvēlēta Python valoda, jo tā ļauj vieglāk īstenot darbā noteiktos mērķus, tomēr dažādas šīs valodas nianses, kā GIL (Global Interpeter Lock) ievērojami palēlināja apmācības un testēšanas procesu, cenšoties tos efektīvi paralelizēt.
8. Šis lēnais apmācības process ierobežo un palēnina arī pašu testēšanas, eksperimentu veikšanas procesu, piemēram, lai pārbaudītu, kā dažādu modifikāciju veikšana neironu tīklā, vai MCTS algoritmā ir ietekmējusi aģenta veikstpēju, tas atkal ir jāapmāca no jauna, lai salīdzinātu veco versiju ar jauno un noskaidrotu, vai ir parādījušies kādi uzlabojumi.
9. Laiks, ko ir iespējams veltīt aģentu apmācībai ir ierobežots sakarā ar to, ka dators apmācības procesā paliek gandrīz pilnībā nelietojams, ņemot vērā to, ka tas gandrīz pilnībā izmanto pieejamos skaitļošanas resursus – visas datoram pieejamās plūsmas tiek izmantotas spēļu ģenerēšanai.

Turpmākie pētījuma virzieni:

1. Spēles stāvokļa kodēšanas uzlabojumi – dažu iepriekšējo gājienu plakņu iekļaušana.

2. Ieviest sarežģītāku heiristiku pašspēles aģentam ar mērķi pilnībā izskaust gājienu atkārtošanos un uzlabot ģenerēto spēļu kvalitāti.
3. Apmācības pieeju apvienošana – sākt apmācību ar eksistējošām spēlēm, to turpinot ar pašspēli.
4. Pāriet no Python programmēšanas valodas uz, piemēram, C++, kas varētu ievērojami paātrināt spēles stāvokļu kodēšanu un eksistējošo spēļu aģenta spēļu ģenerēšanu, novedot pie augstākas veiktspējas aģentiem.
5. 3 galveno arhitekūru (CNN, NNUE, transformatori) salīdzinošā analīze šaha spēles kontekstā, izmantojot abas apmācības metodes.

IZMANTOTIE INFORMĀCIJAS AVOTI

Adrein. *AI in chess: brute force or finesse?* Tiešsaiste. 18.11.2024. Pieejams: https://www.techno-science.net/en/news/ai-in-chess-brute-force-finesse-N26004.html [skatīts 2025-05-05].

Bajaj P., *Reinforcement Learning.* GeeksForGeeks. Tiešsaiste. 24.02.2025. Pieejams: https://www.geeksforgeeks.org/what-is-reinforcement-learning/ [skatīts 2025-03-26]

Brown, S. *Machine Learning, Explained*. Tiešsaiste. MIT Sloan, 2021.04.21. Pieejams: https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained. [skatīts 2025-01-19].

Browne C.B., Powley E., Whitehouse D., Lucas S. M., Cowling P. I., Rohlfshagen P., Tavener S., Perez D., Samothrakis S., Colton S. A survey of monte carlo tree search methods. No: *IEEE Transactions on Computational Intelligence and AI in Games.* Vol. 4, No. 1. 2012., 1.-49. lpp.
Pieejams: https://www.researchgate.net/publication/235985858_A_Survey_of_Monte_Carlo_Tree_Search_Methods.

Calderan F.V. *Minimax Algorithm Applied to Chess Engines.* Tiešsaiste. 12.10.2020. Pieejams: https://fvcalderan.github.io/myworks/articles/minimax_chess_engine.pdf [skatīts 2025-01-20].

Chandan S. *Elo Rating System – Everything You Need to Know*. Tiešsaiste. N.d. Pieejams: https://chessklub.com/elo-rating-system/. [skatīts 2025-02-11].

Cloudflare. *What is a neural network?*. Tiešsaiste. n.d. Pieejams: https://www.cloudflare.com/learning/ai/what-is-neural-network/ [skatīts 2025-05-13].

Computerchess.org. *CCRL 40/15*. Tiešsaiste. Computerchess. 2025-05-19. Pieejams: https://computerchess.org.uk/ccrl/4040/ [skatīts 2025-05-19].

Dey D., *Hierarchical Clustering in Machine Learning.* GeeksForGeeks. Tiešsaiste. 04.02.2025. Pieejams: https://www.geeksforgeeks.org/hierarchical-clustering/ [skatīts 2025-03-24].

Ferrer, J., *How Transformers Work: A Detailed Exploration of Transformer Architecture*. Tiešsaiste. DataCamp. 2024-01-09. Pieejams: https://www.datacamp.com/tutorial/how-transformers-work [skatīts 2025-05-25].

FIDE. *World Top Players.* Tiešsaiste. 02.2025.
Pieejams: https://ratings.fide.com/top_lists.phtml?list=open. [skatīts 2025-02-11].

Filimowicz M. *The Game State, Information & Movement.* Tiešsaiste. Medium, 3.01.2023.
Pieejams: https://medium.com/understanding-games/the-game-state-f57e3e512bf7 [skatīts 2025-01-20].
Georgyan, D. *AlphaZero vs Stockfish 8: A Landmark Battle of Human and Artificial Intelligence in Chess.* Tiešsaiste. Medium, 2023.
Pieejams: https://medium.com/@david_georgyan/alphazero-vs-stockfish-8-a-landmark-battle-of-human-and-artificial-intelligence-in-chess-ac6250b2355c [skatīts 2025-03-05].
Gillis, A. S., *What is a convolutional neural network (CNN)?*. Tiešsaiste. TechTarget. 2024.11. Pieejams: https://www.techtarget.com/searchenterpriseai/definition/convolutional-neural-network [skatīts 2025-05-15].
Glickman M. E., Jones A. C. *Rating the Chess Rating System.* Tiešsaiste. 1999. Pieejams: http://www.glicko.net/research/chance.pdf. [skatīts 2025-02-11].
Gupta M., *What is unsupervised learning?*. Tiešsaiste. GeeksForGeeks. 15.01.2025. Pieejams: https://www.geeksforgeeks.org/unsupervised-learning/ [skatīts 2025-03-24].
Hartikka L. *A step-by-step guide to building a simple chess AI.* Tiešsaiste. 30.03.2017. Pieejams: https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977. [skatīts 2025-02-10].
Heule M. J. H., Rothkrantz L. J. M. *Solving games.* Tiešsaiste. N.d.
Pieejams: https://www.cs.utexas.edu/~marijn/publications/solving_games.pdf [skatīts 2025-01-20].
IBM. *The games that helped AI evolve*. Tiešsaiste. n.d.
Pieejams: https://www.ibm.com/history/early-games. [skatīts 2025-01-19].
IBM, *What are convolutional neural networks?*. Tiešsaiste. n.d. Pieejams: https://www.ibm.com/think/topics/convolutional-neural-networks [skatīts 2025-05-15].
IBM. *What is a neural network?*. Tiešsaiste. 2021.10.06. Pieejams: https://www.ibm.com/think/topics/neural-networks [skatīts 2025-05-10].
International Organization for Standardization (ISO). Artificial Intelligence and Machine Learning. Tiešsaiste. ISO, n.d. Pieejams: https://www.iso.org/artificial-intelligence/machine-learning#toc1. [skatīts 2025-01-20].

International Organization for Standardization (ISO). ISO/IEC 2382-28:1995, *Information technology – Vocabulary – Part 28: Artificial intelligence – Basic concepts and expert systems*.

jayspeakman. *All You Need To Know About The ELO Rating System.* Tiešsaiste. N.d. Pieejams: https://www.houseofstaunton.com/chess-blog/the-elo-rating-system/#h-origins-of-the-elo-rating-system. [skatīts 2025-02-11].

Jo, T. *Machine Learning Foundations: Supervised, Unsupervised, and Advanced Learning*. Tiešsaiste. 1. izd. Cham: Springer International Publishing AG, 2021. 400. lpp. ISBN 9783030658991. Pieejams: https://doi.org/10.1007/978-3-030-65900-4.

Kartik. *Getting Started with Classification.* Tiešsaiste. 2025

Pieejams: https://www.geeksforgeeks.org/getting-started-with-classification/ [skatīts 2025-01-17].

Klein, D. *Neural Networks for Chess. The magic of deep and reinforcement learning revealed.* Tiešsaiste. 11.06.2022. Pieejams: https://arxiv.org/abs/2209.01506 [skatīts 2025-03-20].

Lczero. *Technical Explanation of Leela Chess Zero.* Tiešsaiste. 13.11.2021. Pieejams: *https://lczero.org/dev/wiki/technical-explanation-of-leela-chess-zero/* [skatīts 2025-03-20].

Lczero. *Transformer progress.* Tiešsaiste. 28.02.2024.

Pieejams: https://draft.lczero.org/blog/2024/02/transformer-progress/ [skatīts 2025-05-05].

Lichess.org. *Chess rating systems.* Tiešsaiste. N.d.

Pieejams: https://lichess.org/page/rating-systems?utm_source=chatgpt.com [skatīts 2025-05-12].

Lin, H., Burnes, A., *NVIDIA DLSS 4 Introduces Multi Frame Generation & Enhancements For All DLSS Technologies.* Tiešsaiste. NVIDIA. 2025-01-06. Pieejams: https://www.nvidia.com/en-us/geforce/news/dlss4-multi-frame-generation-ai-innovations/ [skatīts 2025-05-24].

Marckel, O. *Alpha-beta Pruning in Chess Engines.* Tiešsaiste. 2017. Pieejams: https://umm-csci.github.io/senior-seminar/seminars/spring2017/marckel.pdf. [skatīts 2025-02-10].

Merritt R., *What is a transformer model?.* Tiešsaiste. NVIDIA. 2022-03-25. Pieejams: https://blogs.nvidia.com/blog/what-is-a-transformer-model/ [skatīts 2025-05-16].

Mishra, A. *Introduction to Convolution Neural Network.* Tiešsaiste. GeeksForGeeks. 03.04.2025. Pieejams: https://www.geeksforgeeks.org/introduction-convolution-neural-network/ [skatīts 2025-05-14].

Murel J., Kavlakoglu E., *What is reinforcement learning?*. Tiešsaiste. IBM. 25.03.2024. Pieejams: https://www.ibm.com/think/topics/reinforcement-learning [skatīts 2025-03-29].

Official-stockfish. *NNUE.* Tiešsaiste. 2025-03-31. Pieejams: https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html#what-is-nnue [skatīts 2025-05-17].

Official-stockfish. *Stockfish. An open source chess engine.* Tiešsaiste. N.d. Pieejams: https://official-stockfish.github.io/docs/ [skatīts 2025-05-01].

PyTorch. *Start Locally*. Tiešsaiste. n.d.

Pieejams: https://pytorch.org/get-started/locally/ [skatīts 2025-02-18].

Sadmine, Q.A., Husna, A., Müller, M. Stockfish or Leela Chess Zero? A Comparison Against Endgame Tablebases. No: Hartisch, M., Hsueh, CH., Schaeffer, J. *Advances in Computer Games.* ACG 2023. Lecture Notes in Computer Science, vol 14528. Cham: Springer, 2024, pp. 26-35. Pieejams: https://doi.org/10.1007/978-3-031-54968-7_3.

Schulz, A. *Alpha Zero: Comparing "Orangutans and Apples".* Tiešsaiste. 13.12.2017. Pieejams: https://en.chessbase.com/post/alpha-zero-comparing-orang-utans-and-apples [skatīts 2025-05-01].

Shannon C. E. Programming a Computer for Playing Chess. *The Philosophical Magazine.* 1950, Ser.7, Vol. 41, No. 314, 256.-275. lpp.

Shevtekar S., Malpe M, Bhaila M. Analysis of game tree search algorithms using minimax algorithm and alpha-beta pruning. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*. 2022., Volume 8, Issue 6, 330.-331. lpp.

Pieejams: https://ijsrcseit.com/paper/CSEIT1228644.pdf.

Shor, M. *Perfect Information*. Tiešsaiste. Game Theory.net, 15.09.2005. Pieejams: https://www.gametheory.net/dictionary/PerfectInformation.html. [skatīts 2025-01-19].

Silver, A. *Leela Chess Zero: AlphaZero for the PC***.** Tiešsaiste. 04.26.2018.

Pieejams: https://en.chessbase.com/post/leela-chess-zero-alphazero-for-the-pc/1?utm_source=chatgpt.com [skatīts 2025-03-20].

Stockfishchess. *About.* Tiešsaiste. N.d. Pieejams: https://stockfishchess.org/about/ [skatīts 2025-03-05].

Stockfishchess. *Contributors.* Tiešsaiste. 2025. Pieejams: https://tests.stockfishchess.org/contributors [skatīts 2025-05-24].

Stryker, C., Bergmann, D. *What is a transformer model?*. Tiešsaiste. IBM. 2025-03-28. Pieejams: https://www.ibm.com/think/topics/transformer-model [skatīts 2025-05-16].

Sutton, R. S., Barto, A. G. Chapter 1 – Introduction. No: Sutton, R. S., Barto, A. G. *Reinforcement Learning : an introduction.* 2. izdevums. Amerikas Savienotās Valstis: Westchester Publishing services, 2018, 23.-26. lpp. ISBN 9780262039246.

TCEC. *Cup 14.* Tiešsaisste. TCEC. N.d. Pieejams: https://tcec-chess.com/#game=1&round=fl&season=cup14 [skatīts 2025-05-19].

The Click Reader. *Introduction to Convolutional Neural Networks*. Tiešsaiste. N.d. Pieejams; https://www.theclickreader.com/introduction-to-convolutional-neural-networks/ [skatīts 2025-05-15].

Verma, J. *ReLu Function in Python.* Tiešsaiste. DigitalOcean. 2021.08.04. Pieejams: https://www.digitalocean.com/community/tutorials/relu-function-in-python [skatīts 2025-05-13].

Wilkens I., Khoury J. *Minimax vs. Monte Carlo.* Tiešsaiste. 2024. Pieejams: https://kth.diva-portal.org/smash/get/diva2:1887668/FULLTEXT01.pdf [skatīts 2025-01-20].

PIELIKUMI

1. pielikums

Risinājums

Risinājums ir pieejams sekojošajā GitHub repozitorijā - https://github.com/MiksL/COTM